

Notes on the Standard ML Definition

Alex Nelson

August 1, 2024

Mathematics Subject Classification (MSC 2020): 68Q55, 68Q60, 03B70

Draft printing, January 2025.

Copyright © 2025, Alexander M. Nelson.

You may not charge any fee for copies of this book draft.

Compiled: January 25, 2025 at 11:13AM (PST)

This book was typeset by the author using the L^AT_EX typesetting system.

This revision: 5dc5daf. September 19, 2024 11:53am PDT .

Contents

Contents	iii
Preface	vii
1 Introduction	11
2 Syntax of the Core	13
2.1 Reserved words	13
2.2 Special Constants	13
2.3 Comments	15
2.4 Identifiers	15
2.5 Lexical Analysis	16
2.6 Infix operators	17
2.7 Derived forms	18
2.8 Grammar	18
2.9 Syntactic restrictions	21
3 Syntax of Modules	23
3.1 Reserved words	23
3.2 Identifiers	23
3.3 Infix Operators	23
3.4 Grammar for Modules	24
3.5 Grammar for Modules	24
3.6 Syntactic Restrictions	25
4 Static Semantics for the Core	27
4.1 Simple Objects	28
4.2 Compound Objects	28
4.3 Projection, Injection, and Modification	30
4.4 Types and Type Funtions	31
4.5 Type Schemes	32
4.6 Scope of Explicit Type Variables	32
4.7 Non-expansive Expressions	33
4.8 Closure	34
4.9 Type Structures and Type Environments	35
4.10 Inference Rules	36
Atomic Expressions	36
Expression Rows	37
Expressions	37
Matches	38
Match Rules	38
Declarations	38
Value Bindings	40
Type Bindings	40
Datatype Bindings	40

Constructor Bindings	41
Exception Bindings	41
Atomic Patterns	41
Pattern Rows	42
Patterns	42
Type Expressions	42
Type-expression Rows	43
4.11 Further Restrictions	43
5 Static Semantics for Modules	45
5.1 Semantic Objects	45
5.2 Type Realisation	46
5.3 Signature Instantiation	46
5.4 Functor Signature Instantiation	46
5.5 Enrichment	46
5.6 Signature Matching	47
5.7 Inference Rules	47
Structure Expressions	47
Structure-level Declarations	48
Structure Bindings	49
Signature Expressions	49
Signature Declarations	50
Signature Bindings	50
Specifications	50
Value Descriptions	51
Type Descriptions	51
Datatype Descriptions	52
Constructor Descriptions	52
Exception Descriptions	52
Structure Descriptions	52
Functor Declarations	52
Functor Bindings	52
Top-level Declarations	53
6 Dynamic Semantics for the Core	55
6.1 Reduced Syntax	55
6.2 Simple Objects	55
6.3 Compound Objects	56
6.4 Basic Values	56
6.5 Basic Exceptions	57
6.6 Function Closure	57
6.7 Inference Rules	58
Atomic Expressions	59
Expression Rows	60
Expressions	60
Matches	62
Match Rules	62
Declarations	62
Value Bindings	63
Type Bindings	63
Datatype Bindings	64
Constructor Bindings	64
Exception Bindings	64
Atomic Patterns	64

Pattern Rows	65
Patterns	65
7 Dynamic Semantics for Modules	67
7.1 Reduced Syntax	67
7.2 Compound Objects	67
7.3 Inference Rules	69
Structure Expressions	69
Structure-level Declarations	70
Structure Bindings	70
Signature Expressions	70
Signature Declarations	71
Signature Bindings	71
Specifications	71
Value Descriptions	72
Type Descriptions	72
Datatype Descriptions	72
Constructor Descriptions	72
Exception Descriptions	72
Structure Descriptions	72
Functor Bindings	73
Functor Declarations	73
Top-level Declarations	73
8 Programs	75
Programs	76
Bibliography	79
Index	81
Summary Statistics	81

Preface

0.0.1 REMARK: *Work in progress.*

This is a perpetual work in progress. Originally I hoped to learn enough of the Definition of Standard ML to recast it in a more palatable form, but that seems now to be a fool’s errand.

I suspect I will work on this for a while, get bored, abandon it, then revisit it much later on and iterate this process.

For small erections may be finished by their first architects; grand ones, true ones, ever leave the copestone to posterity. God keep me from ever completing anything. This whole book is but a draught—nay, but the draught of a draught. Oh, Time, Strength, Cash, and Patience!

— Herman Melville, *Moby Dick*, Chapter 32 (1851)

0.0.2 REMARK: *Commentary or reconstruction?*

This is an attempt to understand the 1997 Revised Definition [MTHM97], but I am not sure if it should be a “rational reconstruction” (using anachronistic tools and terms to “build from scratch” the contents of the definition) or a “commentary” (trying to articulate the rather terse text).

Milner and Tofte wrote a commentary [MT91] on the original 1990 Definition [MTH90]. It’s arguable how similar the two Definitions are to each other. This makes the usefulness of Milner and Tofte’s commentary subject to debate.

I want to be able to reason about Standard ML code with the results of this document. That’s my goal. Towards that end, I suppose I need a “bit of both”.

*Specifically, this document is geared towards **understanding** the Definition.* So I guess that makes it a “commentary” after all. . .

0.0.3 REMARK: *“The Definition” refers to the 1997 definition.*

We will stop citing the 1997 Revised Definition [MTHM97] and henceforth refer to it as “the Definition” (capitalized). If we want to refer to the 1990 Definition [MTH90], then we will refer to it as “the 1990 Definition” (also capitalized).

When quoting explicitly from the Definition, we will still cite the bibliography entry, as well as provide the pages found in the printed version.

0.0.4 REMARK: *Aside on pedagogical approaches.*

This is probably not the best way to *learn* the Definition. I suspect Milner and Tofte’s Commentary [MT91] has the right approach pedagogically: start with some code snippet, then work through how the Definition elaborates and evaluates the example code. Do this for many different examples.

The ordering and choice in examples is then the difficulty for the author of such a commentary.

0.0.5 REMARK: *Organization of this document.*

This commentary is organized “in parallel” to the Definition. That is to say, the chapters from 2 onwards is commenting on their correspondingly numbered chapters in the 1997 revised Definition.

This contrasts with Milner and Tofte’s Commentary [MT91], which works through the 1990 Definition starting with evaluating expressions, then the static typechecking. Their commentary is probably more enjoyable (and certainly better written) than mine, but I am trying to understand the 1997 Definition completely.

0.0.6 REMARK: *Method of working, species of propositions.*

We will break up the 1997 Definition into atomic propositions, definitions, and inference rules; then we will try to reconstruct what we can from these components. Each atomic proposition is numbered within each section, for ease of reference.

There are different “species” of propositions we offer. Right now, the basic heuristics for their meaning:

- (1) “Deviation” means we are intentionally departing from the definition somehow (e.g., simplifying it by refusing to allow “=” to be a valid identifier that could be defined);
- (2) “Clauses” are rules, constraints, conditions, etc., imposed or required by the Definition; (this term is chosen to be consistent with the terminology found in, e.g., IEEE standards documents — it is not used by the Definition);
- (3) “Rules” are inference rules the Definition gives (plus some of my commentary explaining its significance);
- (4) “Comments” are comments which I am producing for the sake of commenting, clarifying, explicating;
- (5) “Remarks” are... remarks — they are asides, not necessarily shedding light on the meaning of a clause or “ur text”, but comparisons to how other languages do things, or what we’ve learned since the Definition was drafted; you know, things we find... remarkable...;
- (6) “Puzzle” are open questions which, as far as I know, do not have a solution (and I would be very excited to learn more about);
- (7) “Definition” defines a new phrase or term;
- (8) “Convention” usually refers to conventions adopted for metavariables or notation;
- (9) “Example” offers an illustrative example of some concept, definition, or term;
- (10) “Theorem” is self-explanatory;
- (11) “Proposition” is usually a theorem which is not as important;

0.0.7 DEVIATION: *Fragment of Standard ML.*

We will work with the “fragment” of Standard ML which has the following constraints: in the same scope, we cannot redefine a function, constant, whatever.

When we declare a new structure (or signature), we *can* “reuse” an identifier and any references using that identifier refers to the “new” definition.

Any duplicate definitions — or reusing the same identifier in the same scope — for the same structure, type, variable, or data constructor will be flagged as an error.

This is the same constraint that, e.g., Haskell has on redefinitions.

[Used in: 3.6.1, 3.6.6]

0.0.8 PUZZLE: *Formalize the definition using FS_0 ?*

Can we formalize the Definition using Feferman’s FS_0 set theory? Ostensibly, this seems plausible, but I honestly do not know.

Arguably, the more natural metatheory would be some flavor of HOL.

0.0.9 REMARK: *Acknowledgements.*

The formatting of propositions is heavily inspired by Alan U. Kennington’s book on differential geometry [Ken24].

I am also heavily referring to the source of the Definition, which has been made available online for free.¹ I have borrowed some of their formatting macros. The wildcard pattern macro “_” I borrowed from LHS2T_EX (if memory serves me), but the syntactic ellipses “...” as well as the wildcard row pattern “...” I have hacked together.

Many of the comments which Kahrs [Kah93, Kah96] and Rossberg [Ros18b] have made have been integrated into this text, with citations to who observed what.

¹<https://github.com/SMLFamily/The-Definition-of-Standard-ML-Revised/>

INTRODUCTION

1.0.1 COMMENT: *Basic judgements in the Definition.*

In the preface to the 1997 Definition [MTHM97, pp.xi-xii], the authors explain in the preface:

We shall now explain the keystone of our semantic method. First, we need a slight but important refinement. A phrase P is never evaluated *in vacuo* to a meaning M , but always against a background; this background — call it B — is itself a semantic object, being a distillation of the meanings preserved from evaluation of earlier phrases (typically variable declarations, procedure declarations, etc.). In fact evaluation is background-dependent — M depends upon B as well as upon P .

The keystone of the method, then, is a certain kind of assertion about evaluation; it takes the form

$$B \vdash P \Rightarrow M$$

and may be pronounced: “Against the background B , the phrase P evaluates to the meaning M ”. *The formal purpose of this Definition is no more, and no less, than to decree exactly which assertions of this form are true.* This could be achieved in many ways. We have chosen to do it in a structured way, as others have, by giving rules which allow assertions about a *compound* phrase P to be inferred from assertions about its *constituent* phrases $P_1, \dots, P_n$.

Unfortunately, programming language semantics evolved beyond this, and this approach seems archaic.

1.0.2 COMMENT: *Phases to the definition.*

Again, we find in the first chapter of the 1997 Definition, the authors explain (emphasis theirs):

ML is an interactive language, and a *program* consists of a sequence of *top-level declarations*; the execution of each declaration modifies the top-level environment, which we call a *basis*, and reports the modification to the user.

In the execution of a declaration there are three phases: *parsing*, *elaboration*, and *evaluation*. Parsing determines the grammatical form of a declaration. Elaboration, the *static* phase, determines whether it is well-typed and well-formed in other ways, and records relevant type or form information in the basis. Finally evaluation, the *dynamic* phase, determines the value of the declaration and records relevant value information in the basis. Corresponding to these phases, our formal definition divides into three parts: grammatical rules, elaboration rules, and evaluation rules. Furthermore, the basis is divided into the *static* basis and the *dynamic* basis; for example, a variable which has been declared is associated with a type in the static basis and with a value in the dynamic basis.

This gives us a clean way to order our discussion into three phases (parsing, elaboration, and evaluation) and two sublanguages (Core and Module). Since the Definition itself is structured in this manner, we could use that fact to mirror its structure in our text.

CHAPTER 2

SYNTAX OF THE CORE

2.1. RESERVED WORDS

2.1.1 DEFINITION: *Reserved words.*

We are told the following are reserved words in the Core and may not be used as identifiers (except for “=”):

```
abstype  and  andalso  as  case  datatype  do  else
end  exception  fn  fun  handle  if  in  infix
infixr  let  local  nonfix  of  op  open  orelse
raise  rec  then  type  val  with  withtype  while
( )  [ ]  { }  ,  :  ;  ...  _  |  =  =>  ->  #
```

[Used in: 2.5.1]

2.1.2 COMMENT: *Ambiguity about reserved words.*

Kahrs [Kah93] points out that there is ambiguity in the notion of “reserved words”, since there are reserved words *in the Core* and reserved words *in Modules*. The Definition is not clear with what it means by the term (at least, it is not clear to me).

I interpret “reserved words” to refer to those appearing *either* in Core (listed above) *or* in Modules (§3.1.1).

2.2. SPECIAL CONSTANTS

2.2.1 DEFINITION: *Integer constant.*

An “**Integer constant**” in decimal notation is “an optional negation symbol (~) followed by a non-empty sequence of decimal digits 0, ..., 9.” Similarly, in hexadecimal notation, an integer constant is an optional negation symbol followed by 0x followed by a non-empty sequence of hexadecimal digits 0, ..., 9 and a, ..., f (with A, ..., F also allowed).

2.2.2 DEFINITION: *Word constant.*

A “**Word constant**” in decimal notation is 0w followed by a nonempty sequence of decimal digits, and in hexadecimal notation 0wx followed by a non-empty sequence of hexadecimal digits.

2.2.3 REMARK: *Ambiguities about word size.*

The Definition makes no assumptions about the word size, and treats it merely as ‘some kind of unsigned integer’. This is unfortunate. In hindsight, perhaps it should have included a line like “The maximum value for a word is determined by the computer architecture...” or something to that effect.

In practice, SML/NJ tells me, on my 64-bit Intel 2.80 GHz Core 2 Duo T9600 laptop, that `Word.wordSize` is 31 and `SysWord.wordSize` is 32. Examining the output of `cat /proc/cpuinfo`, I find that “address sizes : 36 bits physical, 48 bits virtual”. I have no clue what is going on here practically speaking.

2.2.4 DEFINITION: *Real constant.*

A “**Real Constant**” is an integer constant in decimal notation, possibly followed by a point (.) and one or more decimal digits, possibly followed by an exponent symbol (E or e) and an integer constant in decimal notation; at least one of these optional parts must occur, therefore no integer constant is a real constant.

2.2.5 CLAUSE: *Assumption on the character set.*

The Standard assumes an underlying alphabet of $N \geq 256$ characters numbered 0 to $N-1$, which agrees with the ASCII character set on the characters numbered 0 to 127.

2.2.6 DEFINITION: *Ordinal range.*

The “**Ordinal range**” of the alphabet is the closed interval $[0, N-1]$.

2.2.7 DEFINITION: *String constant.*

A “**String constant**” is a sequence, between quotation marks (") of zero or more printable characters (i.e., numbered 33–126), spaces, or escape sequences.

Each escape sequence starts with the character “\” and stands for a character sequence. The escape sequences are:

<code>\a</code>	A single character interpreted by the system as alert (ASCII 7)
<code>\b</code>	Backspace (ASCII 8)
<code>\t</code>	Horizontal tab (ASCII 9)
<code>\n</code>	Linefeed, also known as newline (ASCII 10)
<code>\v</code>	Vertical tab (ASCII 11)
<code>\f</code>	Form feed (ASCII 12)
<code>\r</code>	Carriage return (ASCII 13)
<code>\^c</code>	The control character c , where c may be any character with number 64–95. The number of <code>\^c</code> is 64 less than the number of c .
<code>\ddd</code>	The single character with number ddd (3 decimal digits denoting an integer in the ordinal range of the alphabet).
<code>\uxxxx</code>	The single character with number $xxxx$ (4 hexadecimal digits denoting an integer in the ordinal range of the alphabet).
<code>\"</code>	"
<code>\\</code>	\
<code>\f···f\</code>	This sequence is ignored, where $f···f$ stands for a sequence of one or more formatting characters.

2.2.8 REMARK: *Unicode string literals.*

The Definition pre-dates the Unicode string encodings and string literals, so we sadly cannot have string literals with emojis. More seriously, we cannot have string literals with mathematical symbols, which is a problem if we’re trying to write a proof assistant (which I am!). For example, you would need to write `"\u00e2\u0088\u0080"` if you wanted to write $\forall$, which...seems horribly unhelpful.¹

2.2.9 DEFINITION: *Formatting characters.*

The “**Formatting Characters**” are a subset of the non-printable characters including at least space, tab, newline, and formfeed. The formfeed allows long strings to be written on more than one line, by writing “\” at the end of one line and at the start of the next.

[Used in: 2.5.2]

¹I learned this from Michael Norrish’s query on the Poly/ML mailing list <https://lists.inf.ed.ac.uk/pipermail/polym1/2008-August/000435.html>

2.2.10 DEFINITION: *Character constants.*

A “**Character constant**” is a sequence of the form `#s`, where *s* is a string constant denoting a string of size one character.

2.2.11 DEFINITION: *Special Constants.*

The Definition denotes by SCon the class of “**Special Constants**”, i.e., the integer, real, word, character, and string constants. The definition uses *scon* to range over SCon.

2.3. COMMENTS**2.3.1** DEFINITION: *Comment.*

A “**Comment**” is any character sequence within comment brackets “`(*`” and “`*)`” in which comment brackets are properly nested.

[Used in: 2.5.2]

2.3.2 CLAUSE: *Space illegal in comment brackets.*

No space is allowed between the two characters which make up a comment bracket — i.e., no space is allowed in “`(*`” nor in “`*)`” if we want them to be comment brackets.

2.3.3 CLAUSE: *Unterminated comments.*

An unmatched “`(*`” should be detected by the compiler.

2.4. IDENTIFIERS**2.4.1** DEFINITION: *Long identifiers, metavariable “longx” ranges over “longX”.*

For each syntactic class X marked as “long”, there is a class “longX” of “**Long Identifiers**”. If *x* ranges over X, then *longx* ranges over longX. The syntax of these long identifiers is given by:

$$\begin{array}{lcl} \text{longx} & ::= & x \\ & | & \text{strid}_1 \cdots \text{strid}_n . x \\ & & (n \geq 1) \end{array}$$

for an identifier and a long identifier, respectively.

The qualified identifiers link the Core and Modules.

[Used in: 2.5.1]

2.4.2 REMARK: *Long identifiers determined by static module semantics.*

The rules governing “When will ‘`val foo = ...`’ be given a long identifier or not?” Well, that’s specified by Chapter 5 the static semantics of modules, specifically structure declarations and structure bindings.

2.4.3 CLAUSE: *Classes of identifiers.*

There are five classes of identifiers:

- (1) Value Identifiers (long) — Vid
- (2) Type Variables — TyVar
- (3) Type Constructors (long) — TyCon
- (4) Lab (record labels) — Lab
- (5) Structure Identifiers (long) — StrId

2.4.4 DEFINITION: *Valid identifiers.*

An identifier is either:

- (1) an “**Alphanumeric identifier**” consisting of any sequence of letters, digits, primes (’), and underbars (‘_’) starting with a letter or a prime; or
- (2) a “**Symbolic identifier**” consisting of any nonempty sequence of the following symbols:

`! % & $ # + - / : < = > ? @ \ ~ ‘ ^ | *`

In both cases, reserved words are excluded.

2.4.5 DEVIATION: *Equality is not a valid identifier.*

We will consciously deviate from the Definition and, consistent with our choices made earlier, insist that “=” is not a valid symbolic identifier. This deviates from the Definition, but it simplifies our life formalizing the Definition.

2.4.6 DEFINITION: *Type Variables.*

A “**Type Variable**” may be any alphanumeric identifier starting with a prime. The class of type variables is denoted “TyVar”.

2.4.7 DEFINITION: *Subclass of equality type variables.*

The subclass of “**Equality Type Variables**” EtyVar of TyVar consists of those which start with two or more primes.

[Used in: 4.1.4]

2.4.8 CONVENTION: *VId, TyCon, Lab do not start with primes.*

The convention therefore is that the classes VId, TyCon, and Lab do not start with primes.

Also, “*” is excluded from TyCon to avoid confusion with derived form of tuple types.

2.4.9 DEFINITION: *Record labels and numeric labels.*

The syntactic class of “**Record Labels**”, denoted Lab, is the alphanumeric identifiers not starting with a prime, extended to include the “**Numeric Labels**” 1, 2, 3, and so on (i.e., any numeral not starting with 0). (This is so tuples can be syntactic sugar as a derived type.)

[Used in: 2.5.1]

2.4.10 DEFINITION: *Structure identifiers.*

The identifier class of “**Structure Identifiers**”, denoted StrId, is represented by alphanumeric identifiers not starting with a prime.

2.4.11 CLAUSE: *Determining class of identifier.*

The class of type variables is disjoint from the other four classes. But determining the class of an identifier *id* in a Core phrase is determined thus:

- (1) Immediately before “.” — i.e., in a long identifier — or in an **open** declaration, *id* is a structure identifier. The rest of the cases assumes *id* is not a structure identifier.
- (2) At the start of a component in a record type, record pattern, or record expression, *id* is a record label.
- (3) Elsewhere in types, *id* is a type constructor.
- (4) Elsewhere, *id* is a value identifier.

2.5. LEXICAL ANALYSIS**2.5.1** CLAUSE: *Items of analysis.*

Each item of lexical analysis is either a reserved word (§2.1.1), a numeric label (§2.4.9), a special constant (Sec 2.2), or a long identifier (§2.4.1). (Remember, an identifier *is* a subclass of long identifiers.)

2.5.2 CLAUSE: *Comments and formatting characters.*

Comments (§2.3.1) and formatting characters (§2.2.9) are separate items (except, of course, within string constants) and are otherwise ignored.

2.5.3 CLAUSE: *Longest next item is always taken.*

At each stage, the longest next item is always taken.

2.6. INFIXED OPERATORS

2.6.1 DEFINITION: *Infix status and infix operators.*

An identifier may be given “**Infix Status**” by the `infix` or `infixr` directive, which may occur as a declaration. This status only pertains to its use as a *vid* within the scope of the directive, and in these uses it is called an “**Infix Operator**”.

2.6.2 CLAUSE: *Qualified identifiers never infix.*

Qualified identifiers **never** have infix status. The Definition is very explicit about this.

2.6.3 EXAMPLE: *Infix operator.*

If *vid* has infix status, then “ $exp_1 \text{ vid } exp_2$ ” (resp., “ $pat_1 \text{ vid } pat_2$ ”) may occur — possibly in parentheses — wherever the application “ $vid\{1=exp_1, 2=exp_2\}$ ” or its derived form “ $vid(exp_1, exp_2)$ ” (resp., “ $vid(pat_1, pat_2)$ ”) would otherwise occur.

2.6.4 CLAUSE: *Op-prefix treats identifier as non-infix.*

An occurrence of *any* long identifier (qualified or not) prefixed by “`op`” is treated as non-infix.

The only required use of an “`op`” is in prefixing a non-infix occurrence of an identifier *vid* which has infix status. Elsewhere “`op`”, where permitted, has no effect.

2.6.5 DEFINITION: *Nonfix directive.*

Infix status is cancelled by the `nonfix` directive.

2.6.6 DEFINITION: *Fixity directives.*

The keywords “`infix`”, “`infixr`”, and “`nonfix`” are collectively referred to as “**Fixity Directives**”.

2.6.7 CLAUSE: *Syntax of fixity directives.*

For $n \geq 1$, we have the syntax for fixity directives look like:

$$\text{infix } \langle d \rangle \text{ vid}_1 \cdots \text{vid}_n$$

$$\text{infixr } \langle d \rangle \text{ vid}_1 \cdots \text{vid}_n$$

$$\text{nonfix vid}_1 \cdots \text{vid}_n$$

where $\langle d \rangle$ is an optional decimal digit d indicating binding precedence. A higher value of d indicates tighter binding; the default is 0.

2.6.8 COMMENT: *Precedence must be between 0 and 9.*

Observe this part of the Definition explicit constrains the precedence for infix operators must be between 0 and 9. In principle, there’s no reason it cannot be something larger (say, representable as an unsigned byte or word — i.e., between 0 and 256, or between 0 and $2^{64} \approx 18.4 \times 10^{18}$).

For what it’s worth, in S4.4.2 of the Haskell 2010 report², it appears that Haskell has the precedence [which it calls the “fixity”] “must be in the range 0 to 9”. So Haskell is following Standard ML here, neat.

2.6.9 CLAUSE: *Associativity of infix operators.*

Here “`infix`” dictates left associativity, “`infixr`” dictates right associativity.

The Definition requires in an expression of the form $exp_1 \text{ vid}_1 \text{ exp}_2 \text{ vid}_2 \text{ exp}_3$ where vid_1 and vid_2 are infix operators with the same precedence, either both must associate to the left or both must associate to the right.

²<http://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-820004.4.2>

2.6.10 EXAMPLE: *Infix operator associativity.*

Suppose “<<” and “>>” are infix operators of equal precedence, but “<<” is left-associative and “>>” is right associative. Then

<code>x << y << z</code>	parses as	<code>(x << y) << z</code>
<code>x >> y >> z</code>	parses as	<code>x >> (y >> z)</code>
<code>x << y >> z</code>	is illegal	
<code>x >> y << z</code>	is illegal	

2.6.11 DEFINITION: *Scope of fixity directive.*

The “**Scope**” of a fixity directive *dir* is the ensuing program text, except if *dir* occurs in a declaration *dec* in either of the phrases

`let dec in ... end`

or

`local dec in ... end`

then the scope of *dir* does not extend beyond these phrases. Further scope limitations are imposed for Modules.

[Used in: 3.3.1]

2.6.12 CLAUSE: *Do not appear in semantic rules.*

The Definition states: These directives and “**op**” are omitted from the semantic rules, since they only affect parsing.

2.7. DERIVED FORMS**2.7.1** CLAUSE: *Syntactic sugar atop the bare language.*

Many syntactic forms in Standard ML can be expressed in terms of a smaller number of syntactic forms which the Definition calls the “**Bare**” language. The derived forms and their equivalents are in Appendix A.

2.8. GRAMMAR**2.8.1** CONVENTION: *Pseudo-EBNF syntax.*

The Definition uses an idiosyncratic EBNF-like syntax. The rules governing it:

- (1) The brackets `< >` enclose optional phrases.
- (2) For any syntax class *X* (over which *x* ranges) we can define the class *Xseq* (over which *xseq* ranges) as follows:

<code>xseq ::=</code>	<code>x</code>	(singleton sequence)
	<code> </code>	(empty sequence)
	<code>(x₁, ..., x_n)</code>	(sequence, $n \geq 1$)

Here the “...” used means syntactic iteration, which is distinct from (and must not be confused with) “...” which is a reserved keyword of the language.

- (3) Alternative forms for each phrase class are in order of decreasing precedence; this resolves ambiguity in parsing (or so the Definition assures us, guiding us to Appendix B)
- (4) L (resp., R) means left (resp., right) association
- (5) Syntax of types binds more tightly than the syntax of expressions
- (6) Each iterated construct (e.g., *match*, ...) extends as far right as possible; therefore, in theory, parentheses may be needed around an expression which terminates with a match (like “**fn match**”) if this occurs within a larger match. (I guess you could say *care must be taken if you play with matches...*)

Unspoken rules:

- (7) “nonterminals” are italicized (*like so*), “terminals” are written in teletype (**like so**).

2.8.2 COMMENT: “*Match extends as far right as possible*” context-free?

I am not sure if this criteria “extends as far right as possible” is context-free, i.e., we might accidentally have a context-sensitive (or worse) grammar accidentally with this seemingly innocuous condition.

Scott, Johnstone, and Walsh [SJW23] have written a paper about this sort of condition. It seems like it is innocent enough, but there may be difficulties or unintended complications with how the Definition uses it.

2.8.3 DEFINITION: *Syntactic classes for Core.*

The syntactic classes for Core, which the Definition calls “**Core Phrase Classes**”, consists of the following:

AtExp	atomic expressions
ExpRow	expression rows
Exp	expressions
Match	matches
Mrule	match rules
Dec	declarations
ValBind	value bindings
TypBind	type bindings
DatBind	datatype bindings
ConBind	constructor bindings
ExBind	exception bindings
AtPat	atomic patterns
PatRow	pattern rows
Pat	patterns
Ty	type expressions
TyRow	type-expression rows

As usual, the convention is to use italicized lowercase names x for metavariables ranging over the syntactic class X .

2.8.4 GRAMMAR: *Syntax for patterns.*

The syntax for patterns:

$atpat$	$::=$	$-$	wildcard
		$scon$	special constant
		$\langle op \rangle longvid$	value identifier
		$\{ \langle patrow \rangle \}$	record
		$\{ pat \}$	
$patrow$	$::=$	$\dots$	wildcard
		$lab = pat \langle , patrow \rangle$	pattern row
pat	$::=$	$atpat$	atomic
		$\langle op \rangle longvid atpat$	constructed pattern
		$pat_1 vid pat_2$	infix value construction
		$pat : ty$	typed
		$\langle op \rangle vid \langle : ty \rangle as pat$	layered

2.8.5 GRAMMAR: *Syntax for types.*

The syntax for types consists of the production rules:

ty	$::=$	$tyvar$	type variable
		$\{ \langle tyrow \rangle \}$	record type expression
		$tyseq longtycon$	type construction
		$ty \rightarrow ty'$	function type expression (R)
		(ty)	
$tyrow$	$::=$	$lab : ty \langle , tyrow \rangle$	type-expression row

2.8.6 GRAMMAR: *Expressions*.

Remember that “special constants” is what the Definition refers to “literal constants”.

<i>atexp</i>	<code>::=</code>	<code>scon</code>	special constant
		<code><op> longvid</code>	value identifier
		<code>{ <exprow> }</code>	record
		<code>let <i>dec</i> in <i>exp</i> end</code>	local declaration
		<code>(<i>exp</i>)</code>	
<i>exprow</i>	<code>::=</code>	<code>lab = <i>exp</i> <, <i>exprow</i></code>	expression row
<i>exp</i>	<code>::=</code>	<code>atexp</code>	atomic
		<code><i>exp</i> <i>atexp</i></code>	application (L)
		<code><i>exp</i>₁ <i>vid</i> <i>exp</i>₂</code>	infix application
		<code><i>exp</i> : <i>ty</i></code>	typed (L)
		<code><i>exp</i> handle <i>match</i></code>	handle exception
		<code>raise <i>exp</i></code>	raise exception
		<code>fn <i>match</i></code>	function

2.8.7 GRAMMAR: “Matches”.

In *case*-expressions, and function declarations (and applications), we have pattern matching based on a set of rules. These are usually called “matches”.

<i>match</i>	<code>::=</code>	<code>mrule < <i>match</i></code>
<i>mrule</i>	<code>::=</code>	<code><i>pat</i> => <i>exp</i></code>

2.8.8 GRAMMAR: *Core declarations*.

This is the heart of what top-level programs look like, they’re a sequence of declarations (of some sort).

<i>dec</i>	<code>::=</code>	<code>val <i>tyvarseq</i> <i>valbind</i></code>	value declaration
		<code>type <i>typbind</i></code>	type declaration
		<code>datatype <i>datbind</i></code>	datatype declaration
		<code>datatype <i>tycon</i> == datatype <i>longtycon</i></code>	datatype replication
		<code>abstype <i>datbind</i> with <i>dec</i> end</code>	abstype declaration
		<code>exception <i>exbind</i></code>	exception declaration
		<code>local <i>dec</i>₁ in <i>dec</i>₂ end</code>	local declaration
		<code>open <i>longstrid</i>₁ ⋯ <i>longstrid</i>_{<i>n</i>}</code>	open declaration (<i>n</i> ≥ 1)
		<code></code>	empty declaration
		<code><i>dec</i>₁ <;> <i>dec</i>₂</code>	sequential declarations
		<code>infix <<i>d</i>> <i>vid</i>₁ ⋯ <i>vid</i>_{<i>n</i>}</code>	infix (L) directive
		<code>infixr <<i>d</i>> <i>vid</i>₁ ⋯ <i>vid</i>_{<i>n</i>}</code>	infix (R) directive
		<code>nonfix <<i>d</i>> <i>vid</i>₁ ⋯ <i>vid</i>_{<i>n</i>}</code>	nonfix directive
<i>valbind</i>	<code>::=</code>	<code><i>pat</i> = <i>exp</i> <and <i>valbind</i>></code>	
		<code>rec <i>valbind</i></code>	
<i>typbind</i>	<code>::=</code>	<code><i>tyvarseq</i> <i>tycon</i> = <i>ty</i> <and <i>typbind</i>></code>	
<i>datbind</i>	<code>::=</code>	<code><i>tyvarseq</i> <i>tycon</i> = <i>conbind</i> <and <i>datbind</i>></code>	
<i>conbind</i>	<code>::=</code>	<code><op> <i>vid</i> <of <i>ty</i>> < <i>datbind</i>></code>	
<i>exbind</i>	<code>::=</code>	<code><op> <i>vid</i> <of <i>ty</i>> <and <i>exbind</i>></code>	
		<code><op> <i>vid</i> = <op> <i>longvid</i> <and <i>exbind</i>></code>	

2.8.9 COMMENT: *Ambiguous grammar for dec*.

Rossberg [Ros18b] observes that the grammar given for *dec* is highly ambiguous. For example, combining the empty declaration *and* sequencing allows the derivation of arbitrary sequences of empty declarations for *any* input.

A second ambiguity Rossberg points out is that a sequence of the form “*dec*₁ *dec*₂ *dec*₃” can be reduced in two ways to *dec*: either via “*dec*₁₂ *dec*₃” or via “*dec*₁ *dec*₂₃”. Rossberg cites Kahrs [Kah93, S8.3] for this second ambiguity, which Kahrs expands upon with several more severe ambiguities.

2.8.10 COMMENT: *Proposed alternative grammar for declarations.*

It seems to me the correct grammar for declarations should be to introduce a syntactic class for ‘atomic declarations’ “*adec*”, another syntactic class for a [possibly semicolon separated, right associative³] sequence of atomic declarations “*adecseq*”, and then just make “*dec*” either empty or an *adecseq*:

<i>adec</i>	::=	<i>val tyvarseq valbind</i>	value declaration
		<i>type typbind</i>	type declaration
		<i>datatype datbind</i>	datatype declaration
		<i>datatype tycon</i> -- <i>datatype longtycon</i>	datatype replication
		<i>abstype datbind with dec end</i>	abstype declaration
		<i>exception exbind</i>	exception declaration
		<i>local dec₁ in dec₂ end</i>	local declaration
		<i>open longstrid₁ ... longstrid_n</i>	open declaration ($n \geq 1$)
		<i>infix</i> $\langle d \rangle$ <i>vid₁ ... vid_n</i>	infix (L) directive
		<i>infixr</i> $\langle d \rangle$ <i>vid₁ ... vid_n</i>	infix (R) directive
		<i>nonfix</i> $\langle d \rangle$ <i>vid₁ ... vid_n</i>	nonfix directive
<i>adecseq</i>	::=	<i>adec</i> $\langle ; \rangle$	a single declaration
		<i>adec</i> $\langle ; \rangle$ <i>adecseq</i>	sequential declarations
<i>dec</i>	::=		empty declaration
		<i>adecseq</i>	sequential declarations

2.9. SYNTACTIC RESTRICTIONS**2.9.1** CLAUSE: *No repeated labels in records.*

No expression row, pattern row, or type-expression row may bind the same *lab* twice.

[Used in: 4.10.45]

2.9.2 CLAUSE: *Cannot bind same identifier twice.*

No binding *valbind*, *typbind*, *datbind*, or *exbind* may bind the same identifier twice. This applies also to value identifiers within a *datbind*.

[Used in: 4.10.30, 4.10.32, 4.10.34, 4.10.35]

2.9.3 CLAUSE: *Type variables sequence uniqueness.*

No *tyvarseq* may contain the same *tyvar* twice. So (*'a*, *'b*, *'a*) is illegal since *'a* appears twice.

[Used in: 4.10.32]

2.9.4 CLAUSE: *rec restriction for derived function form.*

For each value binding “*pat* = *exp*” within **rec**, *exp* must be of the form “**fn** *match*”. The derived form of function-value binding given in Appendix A necessarily obeys this restriction.

2.9.5 CLAUSE: *No binding primitive data constructors.*

No *datbind*, *valbind*, or *exbind* may bind **true**, **false**, **nil**, **::**, or **ref**.

2.9.6 CLAUSE: *No data or exception binding to it.*

No *datbind* or *exbind* may bind **it**.

2.9.7 CLAUSE: *Real constants never appear in patterns.*

No real constant may occur in a pattern. (Really, unless you’re a numerical analyst, you should be staying away from floating-point arithmetic altogether. . .)

³HaMLet parses sequences of declarations as left associative, according to §4.4 of its documentation.

2.9.8 COMMENT: *Problem stems from `real` is not equality type.*

The difficulty with the previous clause (real constants cannot appear in patterns) stems from `real` is not an equality type. This (real not being an equality type) has felt wrong to me, since the IEEE 754 standard *does* define equality for floating-point numbers.

Moreover, it's the *only* type which is not an equality type, and that causes serious complications — it's the entire reason we need to distinguish “equality types” from “non-equality types”! But if we allow equality for real constants, the only possible surprise is that `NaN` is never equal to anything else (i.e., we lose reflexivity).

2.9.9 CLAUSE: *Type variable restrictions in value declarations.*

In a value declaration “`val tyvarseq valbind`”, if *valbind* contains another value declaration “`val tyvarseq' valbind'`”, then *tyvarseq* and *tyvarseq'* must be disjoint.

In other words, no type variable may be scoped by two value declarations of which one occurs inside the other.

This restriction applies after *tyvarseq* and *tyvarseq'* have been extended to include implicitly scoped type variables (as explained in Section 4.6).

[Used in: 4.8.3, 4.10.30]

SYNTAX OF MODULES

3.1. RESERVED WORDS

3.1.1 DEFINITION: *Reserved words.*

The modules have additional reservewords, namely:

```
eqtype    functor    include    sharing    sig
signature  struct    structure  where      :>
```

[Used in: 2.1.2]

3.2. IDENTIFIERS

3.2.1 DEFINITION: *Functor identifier syntactic class FunId.*

The functor identifier syntactic class, denoted FunId, consists of alphanumeric identifiers not starting with a prime.

3.2.2 DEFINITION: *Signature Identifier syntactic class SigId.*

The signature identifier syntactic class, denoted SigId, also consists of alphanumeric identifiers not starting with a prime.

3.2.3 CONVENTION: *Disjointness of identifier classes.*

Henceforth, the Definition considers all identifier classes to be disjoint and discernible by grammatical rules.

3.3. INFIXED OPERATORS

3.3.1 CLAUSE: *Additional scoping rules.*

In addition to the rules governing the scope of infix operators in the Core language (§2.6.11), there is a further scope limitation: if *dir* occurs in a structure-level declaration *strdec* in any of the phrases

```
let strdec in ... end
local strdec in ... end
struct strdec end
```

then the scope of *dir* does not extend beyond the phrase.

An immediate consequence: fixity is local to a basic structure expression (in particular, to such an expression occurring as a functor body).

3.4. GRAMMAR FOR MODULES

3.4.1 DEFINITION: *Syntactic classes for modules.*

The syntactic classes for modules, which the Definition calls “**Module Phrase Classes**”, consists of the following:

StrExp	structure expressions
StrDec	structure-level declarations
StrBind	structure bindings
SigExp	signature expressions
SigDec	signature declarations
SigBind	signature bindings
Spec	specifications
ValDesc	value descriptions
TypDesc	type descriptions
DatDesc	datatype descriptions
ConDesc	constructor descriptions
ExDesc	exception descriptions
StrDesc	structure descriptions
FunDec	functor declarations
FunBind	functor bindings
TopDec	top-level declarations

3.5. GRAMMAR FOR MODULES

3.5.1 GRAMMAR: *Structures.*

The syntax for structures is fairly straightforward. Note that the rule for structure declarations *strdec* includes the rule for core declarations *dec*, which is particularly important later on.

<i>strexp</i>	<code>::=</code>	<code>struct strdec end</code>	basic
		<code>longstrid</code>	structure identifier
		<code>strexp : sigexp</code>	transparent constraint
		<code>strexp :> sigexp</code>	opaque constraint
		<code>funid(strexp)</code>	functor application
		<code>let strdec in strexp end</code>	local declaration
<i>strdec</i>	<code>::=</code>	<code>dec</code>	declaration
		<code>structure strbind</code>	structure
		<code>local strdec₁ in strdec₂ end</code>	local declaration
			empty
		<code>strdec₁ <;> strdec₂</code>	sequential
<i>strbind</i>	<code>::=</code>	<code>strid = strexp <and strbind></code>	

3.5.2 COMMENT: *Ambiguous grammar for strdec.*

Rossberg [Ros18b] notes that the production rule for sequential structure declarations “*strdec₁ <;> strdec₂*” is ambiguous. Is it left-associative or right-associative?

I’d advocate for making it a right recursive production rule, so we could write a simple recursive descent parser for Standard ML.

3.5.3 GRAMMAR: *Signatures.*

The syntax for signatures:

<i>sigexp</i>	<code>::=</code>	<code>sig spec end</code>	basic
		<code>sigid</code>	signature identifier
		<code>sigexp where type tyvarseq longtycon = ty</code>	type realisation

$sigdec ::= \text{signature } sigbind$
 $sigbind ::= sigid = sigexp \langle \text{and } sigbind \rangle$

3.5.4 GRAMMAR: *Specifications.*

The syntax for specifications which appear in the body of a signature:

$spec ::=$

<code>val valdesc</code>	value
<code>type typdesc</code>	type
<code>eqtype typdesc</code>	equality type
<code>datatype datdesc</code>	datatype
<code>datatype tycon == datatype longtycon</code>	replication
<code>exception exdesc</code>	exception
<code>structure strdesc</code>	structure
<code>include sigexp</code>	include
	empty
<code>spec₁ <;> spec₂</code>	sequential
<code>spec sharing type longtycon₁ = ... = longtycon_n</code>	sharing ($n \geq 2$)

 $valdesc ::= vid : ty \langle \text{and } valdesc \rangle$
 $typdesc ::= tyvarseq tycon \langle \text{and } typdesc \rangle$
 $datdesc ::= tyvarseq tycon = condesc \langle \text{and } datdesc \rangle$
 $condesc ::= vid \langle \text{of } ty \rangle \langle | condesc \rangle$
 $exdesc ::= vid \langle \text{of } ty \rangle \langle | exdesc \rangle$
 $strdesc ::= strid : sigexp \langle \text{and } strdesc \rangle$

3.5.5 GRAMMAR: *Functor declarations.*

The syntax for declaring a functor reuses the syntax for structure expressions. Also observe that it is impossible to have a functor with 0 arguments, or with $n > 1$ arguments.

$fund ::= \text{functor } funbind$
 $funbind ::= funid (strid : sigexp) = \text{strexp} \langle \text{and } funbind \rangle$ functor binding

3.5.6 GRAMMAR: *Top-level declarations.*

The syntax for top-level declarations (which forms the basis of a program) consists of a sequence of one or more structure declarations [remember: core-language declarations are swept into structure declarations], signature declarations, and/or functor declarations:

$topdec ::=$

<code>strdec <topdec></code>	structure-level declaration
<code>sigdec <topdec></code>	signature declaration
<code>fundec <topdec></code>	functor declaration

3.5.7 PUZZLE: *LL(1) grammar for Standard ML?*

Can we revise the grammar for Standard ML (both Core and Modules) to make it explicitly LL(1)?

3.6. SYNTACTIC RESTRICTIONS

3.6.1 CLAUSE: *No rebinding the same identifier in simultaneous declarations.*

No binding *strbind*, *sigbind*, or *funbind* may bind the same identifier twice. The Definition means by this, if we have something like “**functor** **foo**(**arg1** : **sig1**) = ... **and** **foo**(**arg2** : **sig2**) = ...” then we have both functor declarations use the same identifier “foo”.

However, our deviation (§0.0.7) already imposes a stronger constraint.

3.6.2 CLAUSE: *Specifications cannot reuse identifiers.*

No description *valdesc*, *typdesc*, *datdesc*, *exdesc*, or *strdesc* may describe the same identifier twice. This also applies to value identifiers within a *datdesc*. (Again, we’re imposing a stronger constraint...)

3.6.3 CLAUSE: *Type variables must be distinct.*

No *tyvarseq* may contain the same *tyvar* twice.

3.6.4 CLAUSE: *Type variables in data constructor appear in type constructor.*

Any *tyvar* appearing on the right side of a *datdesc* of the form “*tyvarseq tycon = ...*” must occur in the *tyvarseq*. Similarly, in signature expressions of the form “*sigexp where type tyvarseq longtycon = ty*”, any *tyvar* occurring in *ty* must occur in *tyvarseq*.

3.6.5 CLAUSE: *Specification identifiers are not reserved words.*

No *datdesc*, *valdesc*, or *exdesc* “may describe” [whatever that means] `true`, `false`, `nil`, `::`, or `ref`.

Further, no *datdesc* or *exdesc* “may describe” `it`.

3.6.6 COMMENT: *Constraining our deviation.*

So our deviation (§0.0.7) from the “full” language imposed the constraint that redefinitions are illegal, but we can “shadow” an identifier if it is inside a structure. We see that the Definition further limits the possible identifiers we can shadow *in general*.

3.6.7 CLAUSE: *Top declaration restriction.*

No *topdec* may contain, as an initial segment, a *strdec* followed by a semicolon. (Rossberg [Ros18b] observes that the *intention* is to make parsing of top-level semicolons unambiguous, so that they always terminate a program.)

[Used in: 8.0.13]

3.6.8 COMMENT: *Semicolons treated worse than a dead dog.*

It seems to me that semicolons are being treated worse than a dead dog, and it might actually improve the language if every statement and declaration had to end with a semicolons. Or if semicolons were banned altogether. It’s got to be one or the other, because right now it’s this muddled middle-of-the-road strategy (which is a great way to get hit by a car).

3.6.9 COMMENT: *References on Standard ML syntax.*

Scott and Jonstone [SJ23] have re-examined Standard ML’s syntax using modern lexical tools.

STATIC SEMANTICS FOR THE CORE

4.0.1 REMARK: *Strategy: introduce “semantic objects”.*

The Definition will introduce what is vaguely described as “semantic objects”, then use these to present the semantics for both Core and Modules, both static and dynamic. We’ll probably need to expand the notion of “semantic object” as needed in the process.

Towards that end, we will have “simple” semantic objects (usually associated with an identifier of some kind) which will eventually combine into “compound” semantic objects (like types or environments).

4.0.2 PUZZLE: *Strength of metatheory?*

These “semantic objects” live in some kind of metatheory or metalanguage. How strong must this metalanguage must be? Presumably HOL suffices (so something like Zermelo set theory also suffices), but would a weaker metatheory work?

4.0.3 REMARK: *Definition describes how to produce trees.*

The Definition describes how to produce (what Milner and Tofte [MT91] call) ‘elaboration trees’ using the static semantics, and ‘evaluation trees’ using the dynamic semantics. This may be surprising to readers familiar with type theory having learned it from, e.g., TAPL where evaluation is just a judgement form described using inference rules.

So be forewarned: the static module chapter describes how to form these elaboration trees, the static core chapter describes one particular form of a “structure declaration”. The dynamic chapters describe analogous results with the evaluation trees. The last chapter describes how to combine the two for a program.

4.0.4 PUZZLE: *Related static semantics to Hindley–Milner type system.*

The Hindley–Milner type system describes a type theory for some polymorphic simply-typed lambda calculus. Some references (like Benjamin Pierce’s TAPL) use a typing judgement with “constraints” to describe the Hindley–Milner type system. (I believe the original paper had the typing judgement look like $\Gamma \vdash t : T, S$ where S is a substitution used in unifying expressions. Pierce turned S into a constraint.) How can we extend this traditional presentation of Hindley–Milner to Standard ML’s static semantics? Is this even the “right way” to look at things?

4.1. SIMPLE OBJECTS

4.1.1 DEFINITION: *Type constructor names.*

The “**Type Constructor Names**” are the values taken by type constructors. The Definition usually refers to them as “type names”, but they must be clearly distinguished from “type variables” and “type constructors”.

4.1.2 CLAUSE: *Simple objects = Identifiers + type constructors + id status.*

All simple objects in static semantics of the language are built from identifiers, type constructor names, and identifier status descriptors. The Definition offers the following table for the possible simple semantic objects:

α or <i>tyvar</i>	$\in$	TyVar	type variables
t	$\in$	TyName	type names
<i>is</i>	$\in$	IdStatus = {c, e, v}	identifier status descriptors

4.1.3 COMMENT: *Type names internally track datatype type operators.*

Just for clarity’s sake, the $t \in \text{TyName}$ are metasymbols which are freshly generated whenever a **datatype** declaration is encountered. This technically differs from a type operator (which is constructed whenever a **type** alias is encountered).

But we can still meaningfully talk about the “arity” of a type name. The Definition allows for eta-equivalence, so $\Lambda\alpha^{(k)}.t$ is replaceable by t .

4.1.4 DEFINITION: *Equality attribute, admitting equality.*

Each $\alpha \in \text{TyVar}$ possesses a Boolean “**Equality**” attribute, which determines whether or not it “**Admits Equality**”, i.e., whether it is a member of EtyVar (§2.4.7)

4.1.5 DEFINITION: *Attributes of type names.*

Each $t \in \text{TyName}$ has an “**Arity**” $k \geq 0$, and also possesses an “**Equality Attribute**”. The Definition denotes the class of type names with arity k by $\text{TyName}^{(k)}$.

4.1.6 DEFINITION: *Type of special constants.*

With each special constant *scon* we associate a “**Type Name**”, denoted $\text{type}(\text{scon})$, which is either **int**, **real**, **word**, **char**, or **string** as indicated by Section 2.2.

4.2. COMPOUND OBJECTS

4.2.1 DEFINITION: *Collection of finite subsets.*

Let A be a set. The Definition uses the notation “**Fin** A ” for the collection of all finite subsets of A .

4.2.2 DEFINITION: *Finite maps.*

Let A, B be sets. Then the Definition denotes by “ $A \xrightarrow{\text{fin}} B$ ” the set of all “**Finite Maps**” (partial functions with finite domain) from A to B .

The domain and range of a finite map f are denoted “**Dom** f ” and “**Ran** f ”, respectively.

A finite map may be written explicitly in the form $\{a_1 \mapsto b_1, \dots, a_k \mapsto b_k\}$, $k \geq 0$; in particular the empty map is $\{\}$.

4.2.3 CONVENTION: *Constructing finite map by set comprehension.*

The Definition uses the notation $\{x \mapsto e ; \phi\}$ — a form of set comprehension — to stand for the finite map f whose domain is the set of values x which satisfy the condition ϕ , and whose value on this domain is given by $f(x) = e$.

[Used in: 4.8.3]

4.2.4 DEFINITION: *Modifying finite maps.*

When f and g are finite maps the map $f + g$, called f “**modified**” by g , is the finite map with domain $\text{Dom } f \cup \text{Dom } g$ and values

$$(f + g)(a) = \begin{cases} g(a) & \text{if } a \in \text{Dom } g \\ f(a) & \text{otherwise.} \end{cases}$$

[Used in: 4.3.6]

4.2.5 REMARK: *Notation for modifying finite maps.*

In my opinion, the notation $f + g$ is unfortunate because addition is commutative¹, but $f + g \neq g + f$ when $\text{Dom } f \cap \text{Dom } g \neq \emptyset$. Although I dislike it, and would never use it in a million years in my own personal work, I am trying to adhere to the Definition’s conventions.

I have seen other people use the notation $f <+ g$ (to indicate that g overwrites f , if there is any on $\text{Dom } f \cap \text{Dom } g \neq \emptyset$). MIZAR uses the notation $f + \cdot g$ (MIZAR code: “f +* g”, in FUNCT_4) for this operation.

4.2.6 CLAUSE: *Compound objects for Core static semantics.*

The compound objects for the static semantics of the Core language are:

$$\begin{aligned} \tau &\in \text{Type} = \text{TyVar} \cup \text{RowType} \cup \text{FunType} \cup \text{ConsType} \\ (\tau_1, \dots, \tau_k) &\text{ or } \tau^{(k)} \in \text{Type}^k \\ (\alpha_1, \dots, \alpha_k) &\text{ or } \alpha^{(k)} \in \text{TyVar}^k \\ \varrho &\in \text{RowType} = \text{Lab} \xrightarrow{\text{fin}} \text{Type} \\ \tau \rightarrow \tau' &\in \text{FunType} = \text{Type} \times \text{Type} \\ \text{ConsType} &= \cup_{k \geq 0} \text{ConsType}^{(k)} \\ \tau^{(k)} t &\in \text{ConsType}^{(k)} = \text{Type}^k \times \text{TyName}^{(k)} \\ \theta \text{ or } \Lambda \alpha^{(k)}. \tau &\in \text{TypeFcn} = \cup_{k \geq 0} \text{TyVar}^k \times \text{Type} \\ \sigma \text{ or } \forall \alpha^{(k)}. \tau &\in \text{TypeScheme} = \cup_{k \geq 0} \text{TyVar}^k \times \text{Type} \\ (\theta, VE) &\in \text{TyStr} = \text{TypeFcn} \times \text{ValEnv} \\ SE &\in \text{StrEnv} = \text{StrId} \xrightarrow{\text{fin}} \text{Env} \\ TE &\in \text{TyEnv} = \text{TyCon} \xrightarrow{\text{fin}} \text{TyStr} \\ VE &\in \text{ValEnv} = \text{VId} \xrightarrow{\text{fin}} \text{TypeScheme} \times \text{IdStatus} \\ E \text{ or } (SE, TE, VE) &\in \text{Env} = \text{StrEnv} \times \text{TyEnv} \times \text{ValEnv} \\ T &\in \text{TyNameSet} = \text{Fin}(\text{TyName}) \\ U &\in \text{TyVarSet} = \text{Fin}(\text{TyVar}) \\ C \text{ or } T, U, E &\in \text{Context} = \text{TyNameSet} \times \text{TyVarSet} \times \text{Env} \end{aligned}$$

Note: the Definition takes $\cup$ to mean the *disjoint union* over semantic object classes. All the defined object classes are understood to be disjoint.

4.2.7 DEFINITION: *Free type names and free type variables.*

For any semantic object A , the Definition denotes by:

- (1) “**tynames** A ” the set of type names, and
- (2) “**tyvars** A ” the set of type variables occurring free in A .

Note that Λ and $\forall$ bind type variables.

¹Except for ordinals, where $1 + \omega = \omega$ but $\omega + 1 = \omega \cup \{\omega\}$.

4.2.8 COMMENT: *Universal quantifier.*

The universal quantifier $\forall$ has a similar meaning as used in System F, it appears. But what TAPL calls “type application” is handled implicitly rule (2).

4.2.9 DEFINITION: *Value identifier status.*

There are several things to note here:

A value environment VE maps value identifiers to a pair consisting of a type scheme and an identifier status. If $VE(vid) = (\sigma, is)$, we say vid “**has status**” is in VE .

An occurrence of a value identifier which is elaborated in VE is referred to as:

- (1) a “**Value Variable**” if its status in VE is v ,
- (2) a “**Value Constructor**” if its status in VE is c , and
- (3) a “**Exception Constructor**” if its status in VE is e .

4.3. PROJECTION, INJECTION, AND MODIFICATION**4.3.1** DEFINITION: *Projection.*

If we have a tuple $x = (x_1, x_2, \dots, x_n)$, then we will often find ourselves writing the projection as “ x_i of x ”. That is to say, the Definition relies on metavariable names to indicate which component is selected.

For example, “the value environment component of E ” is denoted “ VE of E ”

4.3.2 CONVENTION: *Projection of component that’s finite map, applying argument.*

When a tuple contains a finite map, the Definition abuses notation and “applies” the tuple to an argument, again relying on the syntactic class of the argument to determine the relevant function.

For example, $C(tycon)$ means $(TE \text{ of } C)tycon$, and $C(vid)$ means $(VE \text{ of } (E \text{ of } C))(vid)$.

4.3.3 CONVENTION: *Environments applied to long identifiers.*

The Definition extends this notational choice to allow environments to be applied to long identifiers. For example, $longvid = \textcolor{red}{strid}_1 \cdots \textcolor{red}{strid}_k.vid$ then $E(longvid)$ means

$$(VE \text{ of } (SE \text{ of } \cdots (SE \text{ of } (SE \text{ of } E) \textcolor{red}{strid}_1) \textcolor{red}{strid}_2 \cdots) \textcolor{red}{strid}_k)vid.$$

4.3.4 DEFINITION: *Injection.*

Components may be injected into tuple classes; for example “ VE in Env” means the environment $(\{\}, \{\}, VE)$.

4.3.5 COMMENT: *Injections initialize all other components to empty set.*

I think what the Definition means to say is that an injection “ x in y ” amounts to a “constructor” of a new Y tuple whose components are all the empty set *except* the X factor.

4.3.6 CONVENTION: *Modification of environment.*

Extending the notation for modifying finite maps (§4.2.4), the Definition modifies an environment E by an environment E' constructing a new environment $E + E'$, preferring E' but resorting to E when necessary. (Or, overwriting any overlapping data shared by E and E' with E' .) This notation is extended to tuples in general.

Components are often left implicit in modification. For example, $E + VE$ means $E + (\{\}, \{\}, VE)$.

For set components, the Definition means “union” for modification, so that $C + (T, VE)$ means

$$((T \text{ of } C) \cup T, U \text{ of } C, (E \text{ of } C) + VE).$$

(I, uh, don't know how to feel about this since it should be $((T \text{ of } C) \setminus T) \cup T$ instead of $(T \text{ of } C) \cup T$.)

4.3.7 DEFINITION: *Extending a tuple coherently.*

When we modify a context C by an environment E (or a type environment TE) while at the same time extending T of C to include the type names of E (resp., of TE), we do this by defining $C \oplus E$ (resp., $C \oplus TE$) to mean $C + (\text{tynames } E, E)$ (resp., $C + (\text{tynames } TE, TE)$).

4.4. TYPES AND TYPE FUNTIONS

4.4.1 DEFINITION: *Equality type, admitting equality.*

A type τ is an “**Equality Type**” (or “**Admits Equality**”) if it is one of the following:

- (1) α , where α admits equality;
- (2) $\{\text{lab}_1 \mapsto \tau_1, \dots, \text{lab}_n \mapsto \tau_n\}$, where each τ_i admits equality;
- (3) $\tau^{(k)}t$, where t and all members of $\tau^{(k)}$ admit equality;
- (4) $(\tau')\text{ref}$.

4.4.2 CLAUSE: *Type functions.*

A type function $\theta = \Lambda\alpha^{(k)}. \tau$ has arity k ; the bound variables must be distinct.

4.4.3 CLAUSE: *Equating two type functions.*

Two type functions are considered equal if they only differ in their choice of bound variables (alpha-conversion). In particular, the equality attribute has no significance in a bound variable of a type function. The Definitions gives as an example $\Lambda\alpha. \alpha \rightarrow \alpha$ and $\Lambda\beta. \beta \rightarrow \beta$ are equal type functions even if α admits equality but β does not.

[Used in: 4.5.3]

4.4.4 CLAUSE: *Eta conversion for type functions.*

If t has arity k , then we write t to mean $\Lambda\alpha^{(k)}. \alpha^{(k)}t$ (eta-conversion); thus $\text{TyName} \subseteq \text{TypeFcn}$.

4.4.5 DEFINITION: *Equality type function, admits equality.*

We say $\theta = \Lambda\alpha^{(k)}. \tau$ is an “**Equality**” type function, or “**Admits Equality**”, if when the type variables $\alpha^{(k)}$ are chosen to admit equality then τ also admits equality.

4.4.6 CLAUSE: *Application of type function, beta reduction.*

We write the application of a type function θ to a vector $\tau^{(k)}$ of types as $\tau^{(k)}\theta$. If $\theta = \Lambda\alpha^{(k)}. \tau$ we set $\tau^{(k)}\theta = \tau\{\tau^{(k)}/\alpha^{(k)}\}$ (the Definition calls this “beta-conversion”).

4.4.7 CONVENTION: *Notation for substitution.*

We write $\tau\{\theta^{(k)}/t^{(k)}\}$ for the result of substituting type functions $\theta^{(k)}$ for type names $t^{(k)}$ in τ . We assume that all beta-conversions are carried out after substitution, so that for example

$$(\tau^{(k)}t)\{\Lambda\alpha^{(k)}. \tau/t\} = \tau\{\tau^{(k)}/\alpha^{(k)}\}.$$

4.5. TYPE SCHEMES



Type schemes are not fully discussed or formalized explicitly in the Definition. It's just assumed you are familiar with what they mean by them. For the most part, I am relying heavily on Pottier and Rémy [PR05] to make sense of their usage in Standard ML.

4.5.1 DEFINITION: *Generalizes a type.*

A type scheme $\sigma = \forall \alpha^{(k)}. \tau$ “**generalizes**” a type τ' , written $\sigma \succ \tau'$, if $\tau' = \tau\{\tau^{(k)}/\alpha^{(k)}\}$ for some $\tau^{(k)}$, where each member τ_i of $\tau^{(k)}$ admits equality if α_i does.

4.5.2 DEFINITION: *Generalizes a scheme.*

A type scheme $\sigma = \forall \alpha^{(k)}. \tau$ “**generalizes**” $\sigma' = \forall \beta^{(\ell)}. \tau'$, written $\sigma \succ \sigma'$, if $\sigma \succ \tau'$ and $\beta^{(\ell)}$ contains no free type variable of σ .

It can be shown that $\sigma \succ \sigma'$ iff, for all τ'' , whenever $\sigma' \succ \tau''$ then also $\sigma \succ \tau''$.

4.5.3 CLAUSE: *Equality of type schemes.*

Two type schemes σ and σ' are considered equal if they can be obtained from each other by renaming and reordering of bound type variables, and deleting type variables from the prefix which do not occur in the body.

Note: unlike equality of type functions (§4.4.3), here the equality attribute *must be preserved* in renaming. For example $\forall \alpha. \alpha \rightarrow \alpha$ and $\forall \beta. \beta \rightarrow \beta$ are only equal if either both α and β admit equality, or neither does.

4.5.4 CLAUSE: *Types considered as type schemes.*

The Definition considers a type τ to be a type scheme, identifying it with $\forall(). \tau$.

4.5.5 COMMENT: *Type schemes should be thought of as “most general type”?*

I think that type schemes should be considered as the “most general type” for a given expression e . If that expression e is used in a situation which requires a “less general type” (i.e., e 's most general type successfully generalizes the situation's required type), then it typechecks alright.

4.6. SCOPE OF EXPLICIT TYPE VARIABLES

4.6.1 CLAUSE: *Type and datatype bindings explicitly introduce type variables.*

In the Core language, a type binding (*tybind*) or datatype binding (*datbind*) can explicitly introduce type variables whose scope is that binding.

4.6.2 CLAUSE: *Value declarations have optional type variable bindings.*

In a value declaration **val** *tyvarseq* *valbind*, the sequence *tyvarseq* binds type variables: a type variable occurs free in **val** *tyvarseq* *valbind* iff it occurs free in *valbind* and is not in the sequence *tyvarseq*.

However, the Definition states, the explicit binding of type variables at **val** is optional, so the only remaining place in the Core language where type variables occur (which must be dealt with):

- (1) the scope of an explicit type variable occurring in the “: *ty*” of a typed expression or pattern;
- (2) the “**of** *ty*” of an exception binding.

The rest of this section, the Definition considers such free occurrences of type variables only.

4.6.3 DEFINITION: *Scope of explicit type variables.*

Every occurrence of a value declaration is said to “**Scope**” a set of explicit type variables, determined as follows:

- (1) a free type variable is unguarded, or
- (2) a free type variable is implicitly scoped.

4.6.4 DEFINITION: *Unguarded occurrence of type variable.*

A free occurrence of α in a value declaration `val tyvarseq valbind` is said to be “**Unguarded**” if the occurrence is not part of a smaller value declaration within `valbind`. In this case we say that α “**occurs unguarded**” in the value declaration.

4.6.5 DEFINITION: *Implicitly scoped type variable.*

We say that α is “**Implicitly scoped**” at a particular declaration `val tyvarseq valbind` in a program if:

- (1) α occurs unguarded in this value declaration, and
- (2) α does not occur unguarded in any larger value declaration containing the given one.

4.6.6 CLAUSE: *Assume every explicit type variable implicitly scoped at val.*

Henceforth, the Definition assumes that for every value declaration `val tyvarseq ...` occurring in the program, every explicit type variable implicitly scoped at the `val` has been added to `tyvarseq` (subject to the syntactic constraint in Section 2.9).

4.6.7 EXAMPLE: *Type variables with different scopes in value declarations.*

For example, in the two declarations

```
val x = let val id:'a->'a = fn z=>z in id id end
val x = (let val id:'a->'a = fn z=>z in id id end; fn z=>z:'a)
```

the type variable `'a` is scoped differently; they become respectively

```
val x = let val 'a id:'a->'a = fn z=>z in id id end
val 'a x = (let val id:'a->'a = fn z=>z in id id end; fn z=>z:'a)
```

Then, according to the inference rules in Section 4.10 the first example can be elaborated, but the second cannot since `'a` is bound at the outer value declaration leaving no possibility of two different instantiations of the type of `id` in the application `id id`.

4.7. NON-EXPANSIVE EXPRESSIONS**4.7.1** DEFINITION: *Non-expansive expression in a given context.*

An expression is “**Non-expansive**” in context C if, after replacing infix forms by their equivalent prefixed forms, and derived forms by their equivalent forms, it can be generated by the following grammar from the non-terminal *nexp*:

$$\begin{aligned} nexp &::= \textcolor{red}{scon} & nexprow &::= \textcolor{red}{lab} = nexp\langle, nexprow \rangle \\ &\langle \text{op} \rangle longvid & & \\ &\{ \langle nexprow \rangle \} & conexp &::= (conexp\langle : \textcolor{red}{ty} \rangle) \\ &(nexp) & &\langle \text{op} \rangle longvid \\ &conexp nexp & & \\ &nexp : \textcolor{red}{ty} & & \\ &\text{fn } \textcolor{red}{match} & & \end{aligned}$$

Restriction: Within a *conexp*, we require *longvid* $\neq$ `ref` and further that (*is of* $C(\text{longvid})$) $\in \{c, e\}$.

All other expressions are said to be “**Expansive**” in C .

4.7.2 CLAUSE: *Non-expansive expressions are always ‘pure’.*

The Definition informs us that the dynamic evaluation of a non-expansive expression will neither (a) generate an exception, nor (b) extend the domain of memory. But the evaluation of an expansive expression might.

4.7.3 COMMENT: *Non-expansive expressions used in closure of value declarations.*

We should note that the notion of “non-expansive expressions” only matters in the definition of the Closure of a value environment relative to a value binding declaration (§4.8.3). This occurs in let-expressions, which is the value restriction rule.

In effect, this is how let-polymorphism is implemented in Standard ML. I should work through, as an example, how the static semantics determines the typing of the program:

```
fun foo (x : string) : unit =
  let
    fun id y = y
  in
    print (Bool.toString(id true));
    print (id "\nmore stuff\n");
    print x
  end;
```

4.8. CLOSURE

4.8.1 DEFINITION: *Closure of a type with respect to a semantic object.*

Let τ be a type and A a semantic object (usually A is a context C). Then $\text{Clos}_A(\tau)$, the “**closure**” of τ with respect to A , is the type scheme $\forall \alpha^{(k)}. \tau$, where $\alpha^{(k)} = \text{tyvars}(\tau) \setminus \text{tyvars } A$.

4.8.2 DEFINITION: *Total closure.*

The Definition abbreviates the “**total**” closure $\text{Clos}_{\{\}}(\tau)$ to $\text{Clos}(\tau)$.

4.8.3 CLAUSE: *Closure of value environment.*

If the range of a value environment VE contains only types (rather than arbitrary type schemes) we set

$$\text{Clos}_A VE = \{ vid \mapsto (\text{Clos}_A(\tau), is) ; VE(vid) = (\tau, is) \}.$$

Recall the convention for constructing finite maps by set comprehension (§4.2.3), which is done here.

Closing a value environment VE that stems from the elaboration of a value binding *valbind* requires extra care to ensure type security of references and exceptions and correct scoping of explicit type variables. Recall (§2.9.9) that *valbind* is not allowed to bind the same variable twice. Thus, for each $vid \in \text{Dom } VE$ there is a unique *pat* = *exp* in *valbind* which binds vid . If $VE(vid) = (\tau, is)$, let $\text{Clos}_{C, \text{valbind}} VE(vid) = (\forall \alpha^{(k)}. \tau, is)$, where

$$\alpha^{(k)} = \begin{cases} \text{tyvars } \tau \setminus \text{tyvars } C, & \text{if } \text{exp} \text{ is non-expansive in } C; \\ (), & \text{if } \text{exp} \text{ is expansive in } C. \end{cases}$$

[Used in: 4.7.3, 4.10.20]

4.9. TYPE STRUCTURES AND TYPE ENVIRONMENTS

4.9.1 DEFINITION: *Well-formed type structures.*

The definition calls a type structure (θ, VE) “**Well-Formed**” if either $VE = \{\}$ or θ is a type name t . (The latter case arises, with $VE \neq \{\}$, in **datatype** declarations.)

4.9.2 DEFINITION: *Well-formed semantic objects.*

An object or “assembly”² A of semantic objects is “**Well-Formed**” if every type structure occurring in A is well-formed.

4.9.3 DEFINITION: *Type structure respects equality.*

A type structure (t, VE) is said to “**Respect Equality**” if, whenever t admits equality, then either $t = \mathbf{ref}$ (see Appendix C) or, for each $VE(vid)$ of the form $(\forall \alpha^{(k)}.(\tau \rightarrow \alpha^{(k)}t), is)$, the type function $\Lambda \alpha^{(k)}. \tau$ also admits equality.

(This ensures that the equality predicate “=” will be applicable to a constructed value (vid, v) of type $\tau^{(k)}t$ only when it is applicable to the value v itself, whose type is $\tau\{\tau^{(k)}/\alpha^{(k)}\}$.)

4.9.4 DEFINITION: *Type environment respecting equality.*

A type environment TE “**Respects Equality**” if all its type structures do so.

4.9.5 DEFINITION: *Type environment maximizes equality.*

Let TE be a type environment, and let T be the set of type names t such that (t, VE) occurs in TE for some $VE \neq \{\}$. Then TE is said to “**Maximise Equality**” if

- (1) TE respects equality, and also
- (2) if any larger subset of T were to admit equality (without any change in the equality attribute of any type names not in T), then TE would cease to respect equality.

4.9.6 DEFINITION: *Environment for abstract type declarations.*

For any TE of the form

$$TE = \{tycon_i \mapsto (t_i, VE_i) ; 1 \leq i \leq k\},$$

where no VE_i is the empty map, and for any E we define $\text{Abs}(TE, E)$ to be the environment obtained from E and TE as follows (just three easy steps):

- (1) Let $\text{Abs}(TE)$ be the type environment $\{tycon_i \mapsto (t_i, \{\}) ; 1 \leq i \leq k\}$ in which all value environments VE_i have been replaced by the empty map.
- (2) Let $t'_1, \dots, t'_k$ be new distinct type names none of which admit equality.
- (3) Then $\text{Abs}(TE, E)$ is the result of simultaneously substituting t'_i for t_i , $1 \leq i \leq k$, throughout $\text{Abs}(TE) + E$. (The effect of the latter substitution is to ensure that the use of equality on an **abstype** is restricted to the **with** part.)

4.9.7 COMMENT: *$\text{Abs}(TE, E)$ used for abstype declarations only.*

Note that this last definition for $\text{Abs}(TE, E)$ is used only in Rule (19) of the Definition for **abstype** declarations.

²Yeah, this is the first time I think the word “assembly” has been used in the Definition, and I’m as confused by its appearance as you probably are...But they will show up again extensively in the chapter on static semantics of modules.

4.10. INFERENCE RULES

4.10.1 CLAUSE: *Judgement form.*

Each rule of the semantics allows inferences among “sentences” [judgements] of the form

$$A \vdash \textit{phrase} \Rightarrow A'$$

where A is usually a context, $\textit{phrase}$ is a phrase of the Core language, and A' is a semantic object (usually a type or an environment). The Definition suggests reading this as “ $\textit{phrase}$ elaborates to A' in (context) A .”

Some rules have extra hypotheses not of this form, which are called “**Side Conditions**”.

4.10.2 CONVENTION: *First and second options.*

The Definition presents the rules with optional phrases within single angle brackets $\langle \rangle$ which are called “**First Options**”. There are also optional phrases within double angle brackets $\langle\langle \rangle\rangle$ which are called “**Second Options**”. Their usage is governed by the following convention:

In each instance of a rule, the first options must be either all present or all absent; similarly the second options must be either all present or all absent.

4.10.3 CLAUSE: *Intended property of context.*

Although not assumed in our definitions, it is intended that every context $C = T, U, E$ has the property that $\textit{tynames } E \subseteq T$. Thus T may be thought of, loosely, as containing all type names which “have been generated”. It is necessary to include T as a separate component in a context, since $\textit{tynames } E$ may not contain all the type names which have been generated; one reason is that a context $T, \emptyset, E$ is a projection of the basis $B = T, F, G, E$ whose other components F and G could contain other such names — recorded in T but not present in E . Of course, remarks about what “has been generated” are not precise in terms of the semantic rules. But the following precise result may easily be demonstrated:

Let S be a sentence $T, U, E \vdash \textit{phrase} \Rightarrow A$ such that $\textit{tynames } E \subseteq T$, and let S' be a sentence $T', U', E' \vdash \textit{phrase}' \Rightarrow A'$ occurring in a proof of S ; then also $\textit{tynames } E' \subseteq T'$.

4.10.4 COMMENT: *Equation numbers match Definition’s rule numbers.*

Again, for the sake of consistency, the number for the rule found in the Definition is written here as its Equation number.

4.10.5 COMMENT: *Comments swept into the rule.*

Note that I have relocated the “comments” found in the Definition to appear in the relevant rule, when it simplifies the presentation.

Atomic Expressions

$C \vdash \textit{atexp} \Rightarrow \tau$

4.10.6 RULE: *Special constants.*

$$\overline{C \vdash \textit{scon} \Rightarrow \textit{type}(\textit{scon})} \quad (1)$$

4.10.7 RULE: *Value variable.*

Note this simplified three rules (for value variables, value constructors, and exception constants) in the 1990 Definition into a single rule. The second premise, $\sigma \succ \tau$, is new to the 1997 Definition.

$$\frac{C(\textit{longvar}) = (\sigma, \textit{is}) \quad \sigma \succ \tau}{C \vdash \textit{longvar} \Rightarrow \tau} \quad (2)$$

The instantiation of type schemes allows different occurrences of a single *longvid* to assume different types. Note that the identifier status is not used in this rule.

4.10.8 RULE: *Record expressions.*

$$\frac{\langle C \vdash \text{exprow} \Rightarrow \varrho \rangle}{C \vdash \{ \langle \text{exprow} \rangle \} \Rightarrow \{ \} \langle + \varrho \rangle \text{ in Type}} \quad (3)$$

4.10.9 RULE: *Let expressions.*

$$\frac{C \vdash \text{dec} \Rightarrow E \quad C \oplus E \vdash \text{exp} \Rightarrow \tau \quad \text{tynames } \tau \subseteq T \text{ of } C}{C \vdash \text{let } \text{dec} \text{ in } \text{exp} \text{ end} \Rightarrow \tau} \quad (4)$$

The use of $\oplus$, here and elsewhere, ensures that type names generated by the first sub-phrase are different from type names generated by the second sub-phrase. The side condition prevents type names generated by *dec* from escaping outside the local declaration.

4.10.10 RULE: *Parenthetical expression.*

$$\frac{C \vdash \text{exp} \Rightarrow \tau}{C \vdash (\text{exp}) \Rightarrow \tau} \quad (5)$$

Expression Rows

$$\boxed{C \vdash \text{exprow} \Rightarrow \varrho}$$

4.10.11 RULE: *Expression rows.*

$$\frac{C \vdash \text{exp} \Rightarrow \tau \quad \langle C \vdash \text{exprow} \Rightarrow \varrho \rangle}{C \vdash \text{lab} = \text{exp} \langle , \text{exprow} \rangle \Rightarrow \{ \text{lab} \mapsto \tau \} \langle + \varrho \rangle} \quad (6)$$

Expressions

$$\boxed{C \vdash \text{exp} \Rightarrow \tau}$$

4.10.12 RULE: *Atomic.*

$$\frac{C \vdash \text{atexp} \Rightarrow \tau}{C \vdash \text{atexp} \Rightarrow \tau} \quad (7)$$

The relational symbol $\vdash$ is overloaded for all syntactic classes (here atomic expressions and expressions).

4.10.13 RULE: *Application.*

$$\frac{C \vdash \text{exp} \Rightarrow \tau' \rightarrow \tau \quad C \vdash \text{atexp} \Rightarrow \tau'}{C \vdash \text{exp atexp} \Rightarrow \tau} \quad (8)$$

4.10.14 RULE: *Typed.*

$$\frac{C \vdash \text{exp} \Rightarrow \tau \quad C \vdash \text{ty} \Rightarrow \tau}{C \vdash \text{exp} : \text{ty} \Rightarrow \tau} \quad (9)$$

Here τ is determined by C and *ty*. Notice that type variables in *ty* cannot be instantiated in obtaining τ ; thus the expression $1 : 'a$ will not elaborate successfully, nor will the expression $(\text{fn } x \Rightarrow x) : 'a \rightarrow 'b$. The effect of type variables in an explicitly typed expression is to indicate exactly the degree of polymorphism present in the expression.

4.10.15 RULE: *Handle exception.*

$$\frac{C \vdash \text{exp} \Rightarrow \tau \quad C \vdash \text{match} \Rightarrow \text{exn} \rightarrow \tau}{C \vdash \text{exp handle match} \Rightarrow \tau} \quad (10)$$

4.10.16 RULE: *Raise exception.*

$$\frac{C \vdash \text{exp} \Rightarrow \text{exn}}{C \vdash \text{raise exp} \Rightarrow \tau} \quad (11)$$

Note that τ does not occur in the premise; thus a **raise** expression has “arbitrary” type.

4.10.17 RULE: *Function*.

$$\frac{C \vdash \text{match} \Rightarrow \tau}{C \vdash \text{fn match} \Rightarrow \tau} \quad (12)$$

Matches

$$\boxed{C \vdash \text{match} \Rightarrow \tau}$$

4.10.18 RULE: *Match*.

$$\frac{C \vdash \text{mrule} \Rightarrow \tau \quad \langle \langle C \vdash \text{match} \Rightarrow \tau \rangle \rangle}{C \vdash \text{mrule} \langle \mid \text{match} \rangle \Rightarrow \tau} \quad (13)$$

Match Rules

$$\boxed{C \vdash \text{mrule} \Rightarrow \tau}$$

4.10.19 RULE: *Mrule*.

$$\frac{C \vdash \text{pat} \Rightarrow (VE, \tau) \quad C + VE \vdash \text{exp} \Rightarrow \tau' \quad \text{tynames } VE \subseteq T \text{ of } C}{C \vdash \text{pat} \Rightarrow \text{exp} \Rightarrow \tau \rightarrow \tau'} \quad (14)$$

This rule allows new free type variables to enter the context. These new type variables will be chosen, in effect, during the elaboration of *pat* (i.e., in the inference of the first hypothesis). In particular, their choice may have to be made to agree with type variables present in any explicit type expression occurring within *exp* (see rule 9).

Declarations

$$\boxed{C \vdash \text{dec} \Rightarrow E}$$

4.10.20 RULE: *Value Declaration*.

$$\frac{U = \text{tyvars}(\text{tyvarseq}) \quad C + U \vdash \text{valbind} \Rightarrow VE \quad VE' = \text{Clos}_{C, \text{valbind}} VE \quad U \cap \text{tyvars } VE' = \emptyset}{C \vdash \text{val tyvarseq valbind} \Rightarrow VE' \text{ in Env}} \quad (15)$$

Here *VE* will contain types rather than general type schemes. The closure of *VE* allows value identifiers to be used polymorphically, via Rule (2).

The side-condition on *U* ensures that the type variables in *tyvarseq* are bound by the closure operation, if they occur free in the range of *VE*.

On the other hand, if the phrase *val tyvarseq valbind* occurs inside some larger value binding *val tyvarseq' valbind'* then no type variable α listed in *tyvarseq'* will become bound by the $\text{Clos}_{C, \text{valbind}} VE$ operation; for α must be in *U* of *C* and hence excluded from closure by the definition of the closure operation (Section 4.8, (§4.8.3), page 34) since U of *C* $\subseteq$ *tyvars C*.

4.10.21 RULE: *Type declaration*.

$$\frac{C \vdash \text{typbind} \Rightarrow TE}{C \vdash \text{type typbind} \Rightarrow TE \text{ in Env}} \quad (16)$$

4.10.22 RULE: *Datatype declaration*.

$$\frac{C \oplus TE \vdash \text{datbind} \Rightarrow VE, TE \quad \forall (t, VE') \in \text{Ran } TE, t \notin (T \text{ of } C) \quad TE \text{ maximises equality}}{C \vdash \text{datatype datbind} \Rightarrow (VE, TE) \text{ in Env}} \quad (17)$$

The side conditions express that the elaboration of each datatype binding generates new type names and that as many of these new names as possible admit equality. Adding *TE* to the context on the left of the $\vdash$ captures the recursive nature of the binding.

4.10.23 RULE: *Datatype replication.*

$$\frac{C(\text{longtycon}) = (\theta, VE) \quad TE = \{ \text{tycon} \mapsto (\theta, VE) \}}{C \vdash \text{datatype } \text{tycon} \text{ } \text{--} \text{ } \text{datatype } \text{longtycon} \Rightarrow (VE, TE) \text{ in Env}} \quad (18)$$

Note that no new type name is generated (i.e., datatype replication is not generative).

4.10.24 RULE: *Abstype declaration.*

$$\frac{\begin{array}{l} C \oplus TE \vdash \text{datbind} \Rightarrow VE, TE \\ \forall (t, VE') \in \text{Ran } TE, t \notin (T \text{ of } C) \\ C \oplus (VE, TE) \vdash \text{dec} \Rightarrow E \\ TE \text{ maximises equality} \end{array}}{C \vdash \text{abstype } \text{datbind with } \text{dec end} \Rightarrow \text{Abs}(TE, E)} \quad (19)$$

The side conditions express that the elaboration of each datatype binding generates new type names and that as many of these new names as possible admit equality. Adding TE to the context on the left of the $\vdash$ captures the recursive nature of the binding.

4.10.25 RULE: *Exception declaration.*

$$\frac{C \vdash \text{exbind} \Rightarrow VE}{C \vdash \text{exception } \text{exbind} \Rightarrow VE \text{ in Env}} \quad (20)$$

No closure operation is used here, as this would make the type system unsound. Example: `exception E of 'a; val it = (raise E 5) handle E f => f(2)`.

4.10.26 RULE: *Local declaration.*

$$\frac{C \vdash \text{dec}_1 \Rightarrow E_1 \quad C \oplus E_1 \vdash \text{dec}_2 \Rightarrow E_2}{C \vdash \text{local } \text{dec}_1 \text{ in } \text{dec}_2 \text{ end} \Rightarrow E_2} \quad (21)$$

4.10.27 RULE: *Open declaration.*

$$\frac{C(\text{longstrid}_1) = E_1 \quad \cdots \quad C(\text{longstrid}_n) = E_n}{C \vdash \text{open } \text{longstrid}_1 \cdots \text{longstrid}_n \Rightarrow E_1 + \cdots + E_n} \quad (22)$$

4.10.28 RULE: *Empty declaration.*

$$\frac{}{C \vdash \Rightarrow \{\} \text{ in Env}} \quad (23)$$

4.10.29 RULE: *Sequential declaration.*

$$\frac{C \vdash \text{dec}_1 \Rightarrow E_1 \quad C \oplus E_1 \vdash \text{dec}_2 \Rightarrow E_2}{C \vdash \text{dec}_1 \langle ; \rangle \text{dec}_2 \Rightarrow E_1 + E_2} \quad (24)$$

Value Bindings

$$C \vdash \text{valbind} \Rightarrow VE$$

4.10.30 RULE: *Value binding.*

$$\frac{C \vdash \text{pat} \Rightarrow (VE, \tau) \quad C \vdash \text{exp} \Rightarrow \tau \quad \langle C \vdash \text{valbind} \Rightarrow VE' \rangle}{C \vdash \text{pat} = \text{exp} \langle \text{and valbind} \rangle \Rightarrow VE \langle + VE' \rangle} \quad (25)$$

When the option is present we have $\text{Dom } VE \cap \text{Dom } VE' = \emptyset$ by the syntactic restrictions (§§2.9.2, 2.9.9).

4.10.31 RULE: *Recursive value binding.*

$$\frac{C + VE \vdash \text{valbind} \Rightarrow VE \quad \text{tynames } VE \subseteq T \text{ of } C}{C \vdash \text{rec valbind} \Rightarrow VE} \quad (26)$$

Modifying C by VE on the left captures the recursive nature of the binding. From Rule (25) we see that any type scheme occurring in VE will have to be a type. Thus each use of a recursive function in its own body must be assigned the same type. Also note that $C + VE$ may overwrite identifier status. For example, the program `datatype t = f; val rec f = fn x => x;` is legal.

Type Bindings

$$C \vdash \text{typbind} \Rightarrow TE$$

4.10.32 RULE: *Type binding.*

$$\frac{\text{tyvarseq} = \alpha^{(k)} \quad C \vdash \text{ty} \Rightarrow \tau \quad \langle C \vdash \text{typbind} \Rightarrow TE \rangle}{C \vdash \text{tyvarseq tycon} = \text{ty} \langle \text{and typbind} \rangle \Rightarrow \{ \text{tycon} \mapsto (\Lambda \alpha^{(k)}. \tau, \{ \}) \} \langle + TE \rangle} \quad (27)$$

The syntactic restrictions (§§2.9.2–2.9.3) ensure that the type function $\Lambda \alpha^{(k)}. \tau$ satisfies the well-formedness constraint of Section 4.4 and they ensure $\text{tycon} \notin \text{Dom } TE$.

4.10.33 REMARK: *Type operators enter here.*

This is the only way for type operators to appear, it's through type binding rules.

Datatypes introduce new “primitive notions”, as it were: a “type name” which acts like a tagged pair. This resembles a type operator, but does not expand the “tag” part at all. The Definition makes a distinction between type operators [“type functions”] and “type names”, but this seems more like a distinction without a difference.

Datatype Bindings

$$C \vdash \text{datbind} \Rightarrow VE, TE$$

4.10.34 RULE: *Datatype binding.*

$$\frac{\begin{array}{c} \text{tyvarseq} = \alpha^{(k)} \\ C, \alpha^{(k)} t \vdash \text{conbind} \Rightarrow VE \\ \text{arity } t = k \end{array} \quad \langle C \vdash \text{datbind}' \Rightarrow VE', TE' \rangle \quad \forall (t', VE'') \in \text{Ran } TE', t \neq t'}{C \vdash \text{tyvarseq tycon} = \text{conbind} \langle \text{and datbind}' \rangle \Rightarrow (\text{Clos } VE \langle + VE' \rangle, \{ \text{tycon} \mapsto (t, \text{Clos } VE) \} \langle + TE' \rangle)} \quad (28)$$

The syntactic restrictions (§2.9.2) ensure $\text{Dom } VE \cap \text{Dom } VE' = \emptyset$ and $\text{tycon} \notin \text{Dom } TE'$.

NOTE: there is a typo in the Definition, namely there is no closing right parentheses in the conclusion to this rule.

Constructor Bindings

$$\boxed{C, \tau \vdash \text{conbind} \Rightarrow VE}$$

4.10.35 RULE: *Data constructors.*

$$\frac{\langle C \vdash \text{ty} \Rightarrow \tau' \rangle \quad \langle \langle C, \tau \vdash \text{conbind} \Rightarrow VE \rangle \rangle}{C, \tau \vdash \text{vid} \langle \text{of } \text{ty} \rangle \langle \langle \mid \text{conbind} \rangle \rangle \Rightarrow \{ \text{vid} \mapsto (\tau, \text{c}) \} \langle + \{ \text{vid} \mapsto (\tau' \rightarrow \tau, \text{c}) \} \rangle \langle \langle + VE \rangle \rangle} \quad (29)$$

By the syntactic restrictions (§2.9.2) $\text{vid} \notin \text{Dom } VE$.

Exception Bindings

$$\boxed{C \vdash \text{exbind} \Rightarrow VE}$$

4.10.36 RULE: *Exception binding.*

$$\frac{\langle C \vdash \text{ty} \Rightarrow \tau \rangle \quad \langle \langle C \vdash \text{exbind} \Rightarrow VE \rangle \rangle}{C \vdash \text{vid} \langle \text{of } \text{ty} \rangle \langle \langle \text{and } \text{exbind} \rangle \rangle \Rightarrow \{ \text{vid} \mapsto (\text{exn}, \text{e}) \} \langle + \{ \text{vid} \mapsto (\tau \rightarrow \text{exn}, \text{e}) \} \rangle \langle \langle + VE \rangle \rangle} \quad (30)$$

Comments:

- (1) Notice that τ may contain type variables.
- (2) For each C and exbind , there is at most one VE satisfying $C \vdash \text{exbind} \Rightarrow VE$.

4.10.37 RULE: *Exception binding.*

$$\frac{C(\text{longvid}) = (\tau, \text{e}) \quad \langle C \vdash \text{exbind} \Rightarrow VE \rangle}{C \vdash \text{vid} = \text{longvid} \langle \text{and } \text{exbind} \rangle \Rightarrow \{ \text{vid} \mapsto (\tau, \text{e}) \} \langle + VE \rangle} \quad (31)$$

For each C and exbind , there is at most one VE satisfying $C \vdash \text{exbind} \Rightarrow VE$.

Atomic Patterns

$$\boxed{C \vdash \text{atpat} \Rightarrow (VE, \tau)}$$

4.10.38 RULE: *Wildcard pattern.*

$$\overline{C \vdash _ \Rightarrow (\{\}, \tau)} \quad (32)$$

4.10.39 RULE: *Special constant in pattern.*

$$\overline{C \vdash \text{scon} \Rightarrow (\{\}, \text{type}(\text{scon}))} \quad (33)$$

4.10.40 RULE: *Variable pattern.*

$$\frac{\text{vid} \notin \text{Dom}(C) \text{ or } \text{is of } C(\text{vid}) = \mathbf{v}}{C \vdash \text{vid} \Rightarrow (\{ \text{vid} \mapsto (\tau, \mathbf{v}) \}, \tau)} \quad (34)$$

The context C determines whether to apply this rule or the next rule. Note that vid can assume a type, not a general type scheme.

4.10.41 RULE: *Constant pattern.*

$$\frac{C(\text{longvid}) = (\sigma, \text{is}) \quad \text{is} \neq \mathbf{v} \quad \sigma \succ \tau^{(k)}t}{C \vdash \text{longvid} \Rightarrow (\{\}, \tau^{(k)}t)} \quad (35)$$

The context C determines whether to apply this rule or the previous rule.

4.10.42 RULE: *Record pattern.*

$$\frac{\langle C \vdash \text{patrow} \Rightarrow (VE, \varrho) \rangle}{C \vdash \{ \langle \text{patrow} \rangle \} \Rightarrow (\{ \} \langle + VE \rangle, \{ \} \langle + \varrho \rangle \text{ in Type })} \quad (36)$$

4.10.43 RULE: *Parenthesised pattern.*

$$\frac{C \vdash \text{pat} \Rightarrow (VE, \tau)}{C \vdash (\text{pat}) \Rightarrow (VE, \tau)} \quad (37)$$

Pattern Rows

$$\boxed{C \vdash \text{patrow} \Rightarrow (VE, \varrho)}$$

4.10.44 RULE: *Wildcard record.*

$$\overline{C \vdash \dots \Rightarrow (\{\}, \varrho)} \quad (38)$$

4.10.45 RULE: *Record component.*

$$\frac{\frac{C \vdash \text{pat} \Rightarrow (VE, \tau)}{\langle C \vdash \text{patrow} \Rightarrow (VE', \varrho) \quad \text{Dom } VE \cap \text{Dom } VE' = \emptyset \rangle}}{C \vdash \text{lab} = \text{pat} \langle \text{ , patrow} \rangle \Rightarrow (VE \langle + VE' \rangle, \{\text{lab} \mapsto \tau\} \langle + \varrho \rangle)} \quad (39)$$

The syntactic restrictions (§2.9.1) ensure $\text{lab} \notin \text{Dom } \varrho$.

Patterns

$$\boxed{C \vdash \text{pat} \Rightarrow (VE, \tau)}$$

4.10.46 RULE: *Atomic pattern.*

$$\frac{C \vdash \text{atpat} \Rightarrow (VE, \tau)}{C \vdash \text{atpat} \Rightarrow (VE, \tau)} \quad (40)$$

4.10.47 RULE: *Construction pattern.*

$$\frac{C(\text{longvid}) = (\sigma, is) \quad is \neq \mathbf{v} \quad \sigma \succ \tau' \rightarrow \tau \quad C \vdash \text{atpat} \Rightarrow (VE, \tau')}{C \vdash \text{longvid atpat} \Rightarrow (VE, \tau)} \quad (41)$$

4.10.48 RULE: *Typed pattern.*

$$\frac{C \vdash \text{pat} \Rightarrow (VE, \tau) \quad C \vdash \text{ty} \Rightarrow \tau}{C \vdash \text{pat} : \text{ty} \Rightarrow (VE, \tau)} \quad (42)$$

4.10.49 RULE: *Layered pattern.*

$$\frac{\text{vid} \notin \text{Dom}(C) \text{ or } is \text{ of } C(\text{vid}) = \mathbf{v} \quad \langle C \vdash \text{ty} \Rightarrow \tau \rangle \quad C \vdash \text{pat} \Rightarrow (VE, \tau) \quad \text{vid} \notin \text{Dom } VE}{C \vdash \text{vid} \langle : \text{ty} \rangle \text{ as pat} \Rightarrow (\{\text{vid} \mapsto (\tau, \mathbf{v})\} + VE, \tau)} \quad (43)$$

Type Expressions

$$\boxed{C \vdash \text{ty} \Rightarrow \tau}$$

4.10.50 RULE: *Type variable.*

The name used in the L^AT_EX comments for the Definition call this the “atype variable” rule — I’m uncertain why “atype” is used (atomic type? Or because α in Standard ML is ‘a as a type variable?)

$$\frac{\text{tyvar} = \alpha}{C \vdash \text{tyvar} \Rightarrow \alpha} \quad (44)$$

4.10.51 RULE: *Record type.*

$$\frac{\langle C \vdash \text{tyrow} \Rightarrow \varrho \rangle}{C \vdash \{ \langle \text{tyrow} \rangle \} \Rightarrow \{\} \langle + \varrho \rangle \text{ in Type}} \quad (45)$$

4.10.52 RULE: *Constructed type*.

$$\frac{\text{tyseq} = \text{ty}_1 \cdots \text{ty}_k \quad C \vdash \text{ty}_i \Rightarrow \tau_i \ (1 \leq i \leq k) \quad C(\text{longtycon}) = (\theta, \text{VE})}{C \vdash \text{tyseq longtycon} \Rightarrow \tau^{(k)}\theta} \quad (46)$$

Recall that for $\tau^{(k)}\theta$ to be defined, θ must have arity k .

4.10.53 RULE: *Function type*.

$$\frac{C \vdash \text{ty} \Rightarrow \tau \quad C \vdash \text{ty}' \Rightarrow \tau'}{C \vdash \text{ty} \rightarrow \text{ty}' \Rightarrow \tau \rightarrow \tau'} \quad (47)$$

4.10.54 RULE: *Parenthesised type*.

$$\frac{C \vdash \text{ty} \Rightarrow \tau}{C \vdash (\text{ty}) \Rightarrow \tau} \quad (48)$$

Type-expression Rows

$$\boxed{C \vdash \text{tyrow} \Rightarrow \varrho}$$

4.10.55 RULE: *Record type components*.

$$\frac{C \vdash \text{ty} \Rightarrow \tau \quad \langle C \vdash \text{tyrow} \Rightarrow \varrho \rangle}{C \vdash \text{lab} : \text{ty} \langle \text{ , tyrow} \rangle \Rightarrow \{\text{lab} \mapsto \tau\} \langle + \varrho \rangle} \quad (49)$$

The syntactic constraints ensure $\text{lab} \notin \text{Dom } \varrho$.

4.11. FURTHER RESTRICTIONS

4.11.1 COMMENT: *Additional impositions*.

In addition to the inference rules given above, there are three restrictions the Definition imposes. These all concern pattern matching, which would be tedious to write down using inference rules (and I, at least, would find them confusingly misplaced among typing rules).

4.11.2 CLAUSE: *Record pattern must determine uniquely the set of labels*.

For each occurrence of a record pattern containing a record wildcard (i.e., of the form $\{\text{lab}_1 = \text{pat}_1, \dots, \text{lab}_m = \text{pat}_m, \dots\}$) the program context must determine uniquely the domain $\{\text{lab}_1, \dots, \text{lab}_n\}$ of its row type, where $m \leq n$; thus, the context must determine the labels $\{\text{lab}_{m+1}, \dots, \text{lab}_n\}$ of the fields to be matched by the wildcard. For this purpose, an explicit type constraint may be needed.

4.11.3 CLAUSE: *Irredundant patterns*.

In a match of the form $\text{pat}_1 \Rightarrow \text{exp}_1 \mid \dots \mid \text{pat}_n \Rightarrow \text{exp}_n$ the pattern sequence $\text{pat}_1, \dots, \text{pat}_n$ should be *irredundant*; that is, each pat_j must match some value (of the right type) which is not matched by pat_i for any $i < j$.

In the context **fn match**, the *match* must also be *exhaustive*; that is, every value (of the right type) must be matched by some pat_i .

The compiler must give warning on violation of these restrictions, but should still compile the match.

The restrictions are inherited by derived forms; in particular, this means that in the function-value binding $\text{vid } \text{atpat}_1 \cdots \text{atpat}_n \langle \text{ty} \rangle = \text{exp}$ (consisting of one clause only), each separate atpat_i should be exhaustive by itself.

4.11.4 CLAUSE: *Warn about nonexhaustive patterns, but still compile*.

For each value binding $\text{pat} = \text{exp}$ the compiler must issue a report (but still compile) if pat is not exhaustive. This will detect a mistaken declaration like $\text{val nil} = \text{exp}$ in which the user expects to declare a new variable **nil** (whereas

the language dictates that `nil` is here a constant pattern, so no variable gets declared). However, this warning should not be given when the binding is a component of a top-level declaration `val valbind`; e.g. `val x::l = exp1 and y = exp2` is not faulted by the compiler at top level, but may of course generate a `Bind` exception (see Section 6.5).

[Used in: 6.5.2]

STATIC SEMANTICS FOR MODULES

5.1. SEMANTIC OBJECTS

5.1.1 CLAUSE: *Simple semantic objects.*

The simple semantic objects for Modules static semantics are exactly those for the Core from Section 4.1.

5.1.2 DEFINITION: *Compound semantic objects.*

The compound objects are those for Core (Section 4.2), augmented by the following:

$$\begin{aligned}
 \Sigma \text{ or } (T)E &\in \text{Sig} = \text{TyNameSet} \times \text{Env} \\
 \Phi \text{ or } (T)(E, (T')E') &\in \text{FunSig} = \text{TyNameSet} \times (\text{Env} \times \text{Sig}) \\
 G &\in \text{SigEnv} = \text{SigId} \xrightarrow{\text{fn}} \text{Sig} \\
 F &\in \text{FunEnv} = \text{FunId} \xrightarrow{\text{fn}} \text{FunSig} \\
 B \text{ or } T, F, G, E &\in \text{Basis} = \text{TyNameSet} \times \text{FunEnv} \times \text{SigEnv} \times \text{Env}
 \end{aligned}$$

The prefix (T) , in signatures and functor signatures, binds type names.

[Used in: 8.0.5]

5.1.3 CLAUSE: *Projection, injection, modification.*

These operations are analogous ‘in the obvious way’ to how they were defined for Core’s static semantics.

5.1.4 DEFINITION: *Context of basis.*

The Definition introduces the notation $C \text{ of } B$ to be the context $(T \text{ of } B, \emptyset, E \text{ of } B)$, i.e. with an empty set of explicit type variables.

5.1.5 DEFINITION: *Modifying a basis by an environment.*

We frequently need to modify a basis B by an environment E (or a structure environment SE say), at the same time extending T of B to include the type names. We therefore define $B \oplus SE$, for example, to mean $B + (\text{tynames } SE, SE)$.

5.1.6 CLAUSE: *Structures not represented by semantic objects.*

There is no separate kind of semantic object to represent structures: structure expressions elaborate to environments, just as structure-level declarations do. Thus, notions which are commonly associated with structures (for example the notion of matching a structure against a signature) are defined in terms of environments.

5.2. TYPE REALISATION

5.2.1 DEFINITION: *Type realisation, or simply just a “realisation”.*

We define a **“Type Realisation”** is a map $\varphi : \text{TyName} \rightarrow \text{TypeFcn}$ such that t and $\varphi(t)$ have the same arity, and if t admits equality then so does $\varphi(t)$.

5.2.2 DEFINITION: *Support of a type realisation.*

The **“Support”** $\text{Supp } \varphi$ of a type realisation φ is the set of type names t for which $\varphi(t) \neq t$.

5.2.3 DEFINITION: *Yield of a type realisation.*

The **“Yield”** $\text{Yield } \varphi$ of a realisation φ is the set of type names which occur in some $\varphi(t)$ for which $t \in \text{Supp } \varphi$.

5.2.4 CLAUSE: *Extend realisations to all semantic objects.*

Realisations φ are extended to apply to all semantic objects; their effect is to replace each name t by $\varphi(t)$. In applying φ to an object with bound names, such as a signature $(T)E$, first bound names must be changed so that, for each binding prefix (T) ,

$$T \cap (\text{Supp } \varphi \cup \text{Yield } \varphi) = \emptyset.$$

5.3. SIGNATURE INSTANTIATION

5.3.1 DEFINITION:

An environment E_2 is an **“Instance”** of a signature $\Sigma_1 = (T_1)E_1$, written $\Sigma_1 \geq E_2$, if there exists a realisation φ such that $\varphi(E_1) = E_2$ and $\text{Supp } \varphi \subseteq T_1$.

5.4. FUNCTOR SIGNATURE INSTANTIATION

5.4.1 DEFINITION:

A pair $(E, (T')E')$ is called a **“Functor Instance”**. Given $\Phi = (T_1)(E_1, (T'_1)E'_1)$, a functor instance $(E_2, (T'_2)E'_2)$ is an **“Instance”** of Φ , written $\Phi \geq (E_2, (T'_2)E'_2)$, if there exists a realisation φ such that $\varphi(E_1, (T'_1)E'_1) = (E_2, (T'_2)E'_2)$ and $\text{Supp } \varphi \subseteq T_1$.

5.5. ENRICHMENT

5.5.1 CLAUSE:

In matching an environment to a signature, the environment will be allowed both to have more components, and to be more polymorphic, than (an instance of) the signature. More precisely, we define enrichment of environments and type structures recursively as follows:

5.5.2 DEFINITION: *Environment enrichment.*

An environment $E_1 = (SE_1, TE_1, VE_1)$ **“Enriches”** another environment $E_2 = (SE_2, TE_2, VE_2)$, written $E_1 \succ E_2$, if

- (1) $\text{Dom } SE_1 \supseteq \text{Dom } SE_2$, and $SE_1(\text{strid}) \succ SE_2(\text{strid})$ for all $\text{strid} \in \text{Dom } SE_2$
- (2) $\text{Dom } TE_1 \supseteq \text{Dom } TE_2$, and $TE_1(\text{tycon}) \succ TE_2(\text{tycon})$ for all $\text{tycon} \in \text{Dom } TE_2$
- (3) $\text{Dom } VE_1 \supseteq \text{Dom } VE_2$, and $VE_1(\text{vid}) \succ VE_2(\text{vid})$ for all $\text{vid} \in \text{Dom } VE_2$,
where $(\sigma_1, is_1) \succ (\sigma_2, is_2)$ means $\sigma_1 \succ \sigma_2$ and

$$is_1 = is_2 \quad \text{or} \quad is_2 = \text{v}$$

5.5.3 DEFINITION: *Type structure enrichment.*

Finally, a type structure (θ_1, VE_1) **“Enriches”** another type structure (θ_2, VE_2) , written $(\theta_1, VE_1) \succ (\theta_2, VE_2)$, if

- (1) $\theta_1 = \theta_2$
- (2) Either $VE_1 = VE_2$ or $VE_2 = \{\}$.

5.6. SIGNATURE MATCHING

5.6.1 DEFINITION: *Environment matches a signature.*

An environment E “**Matches**” a signature Σ_1 if there exists an environment E^- such that $\Sigma_1 \geq E^- \prec E$. Thus matching is a combination of instantiation and enrichment. There is at most one such E^- , given Σ_1 and E .

5.7. INFERENCE RULES

5.7.1 CLAUSE: *Judgement form for static semantics of Modules.*

The rules of the Modules static semantics allow sentences of the form

$$A \vdash \text{phrase} \Rightarrow A'$$

to be inferred, where in this case A is either a basis, a context or an environment and A' is a semantic object. The convention for options is as in the Core semantics.

5.7.2 CLAUSE: *Assumption about basis for top-level declarations.*

Although not assumed in our definitions, it is intended that every basis $B = T, F, G, E$ in which a *topdec* is elaborated has the property that $\text{tynames } F \cup \text{tynames } G \cup \text{tynames } E \subseteq T$.

5.7.3 THEOREM: *Static semantics preserves the assumption.*

Let S be an inferred sentence $B \vdash \text{topdec} \Rightarrow B'$ in which $B = T, F, G, E$ satisfies the condition $\text{tynames } F \cup \text{tynames } G \cup \text{tynames } E \subseteq T$. Then B' also satisfies the condition.

Moreover, if S' is a sentence of the form $B'' \vdash \text{phrase} \Rightarrow A$ occurring in a proof of S , where *phrase* is any Modules phrase, then B'' also satisfies the condition.

Finally, if $T, U, E \vdash \text{phrase} \Rightarrow A$ occurs in a proof of S , where *phrase* is a phrase of Modules or of the Core, then $\text{tynames } E \subseteq T$.

Structure Expressions

$$B \vdash \text{strexpr} \Rightarrow E$$

5.7.4 RULE: *Generative structure expression.*

$$\frac{B \vdash \text{strdec} \Rightarrow E}{B \vdash \text{struct } \text{strdec} \text{ end} \Rightarrow E} \quad (50)$$

5.7.5 RULE: *Long structure identifier.*

$$\frac{B(\text{longstrid}) = S}{B \vdash \text{longstrid} \Rightarrow S} \quad (51)$$

5.7.6 RULE: *Transparent constraint rule.*

$$\frac{B \vdash \text{strexpr} \Rightarrow E \quad B \vdash \text{sigexpr} \Rightarrow \Sigma \quad \Sigma \geq E' \prec E}{B \vdash \text{strexpr} : \text{sigexpr} \Rightarrow E'} \quad (52)$$

5.7.7 RULE: *Opaque constraint rule.*

$$\frac{\begin{array}{l} B \vdash \text{strexpr} \Rightarrow E \\ B \vdash \text{sigexpr} \Rightarrow (T')E' \\ (T')E' \geq E'' \prec E \\ T' \cap (T \text{ of } B) = \emptyset \end{array}}{B \vdash \text{strexpr} : \text{sigexpr} \Rightarrow E'} \quad (53)$$

5.7.8 RULE: *Functor application.*

$$\frac{\begin{array}{c} B \vdash \text{strexp} \Rightarrow E \\ B(\text{funid}) \geq (E'', (T')E'), E \succ E'' \\ (\text{tynames } E \cup T \text{ of } B) \cap T' = \emptyset \end{array}}{B \vdash \text{funid } (\text{strexp}) \Rightarrow E'} \quad (54)$$

The side condition $(\text{tynames } E \cup T \text{ of } B) \cap T' = \emptyset$ can always be satisfied by renaming bound names in $(T')E'$; it ensures that the generated datatypes receive new names.

Let $B(\text{funid}) = (T)(E_f, (T')E'_f)$. Let φ be a realisation such that $\varphi(E_f, (T')E'_f) = (E'', (T')E')$. Sharing between argument and result specified in the declaration of the functor *funid* is represented by the occurrence of the same name in both E_f and E'_f , and this repeated occurrence is preserved by φ , yielding sharing between the argument structure E and the result structure E' of this functor application.

5.7.9 RULE: *Let-expression in structures.*

$$\frac{B \vdash \text{strdec} \Rightarrow E_1 \quad B \oplus E_1 \vdash \text{strexp} \Rightarrow E_2}{B \vdash \text{let } \text{strdec} \text{ in } \text{strexp} \text{ end} \Rightarrow E_2} \quad (55)$$

The use of $\oplus$, here and elsewhere, ensures that type names generated by the first sub-phrase are distinct from names generated by the second sub-phrase.

Structure-level Declarations

$$\boxed{B \vdash \text{strdec} \Rightarrow E}$$

5.7.10 RULE: *Core declaration.*

$$\frac{C \text{ of } B \vdash \text{dec} \Rightarrow E}{B \vdash \text{dec} \Rightarrow E} \quad (56)$$

5.7.11 RULE: *Structure declaration.*

$$\frac{B \vdash \text{strbind} \Rightarrow SE}{B \vdash \text{structure } \text{strbind} \Rightarrow SE \text{ in Env}} \quad (57)$$

5.7.12 RULE: *Local structure-level declaration.*

$$\frac{B \vdash \text{strdec}_1 \Rightarrow E_1 \quad B \oplus E_1 \vdash \text{strdec}_2 \Rightarrow E_2}{B \vdash \text{local } \text{strdec}_1 \text{ in } \text{strdec}_2 \text{ end} \Rightarrow E_2} \quad (58)$$

5.7.13 RULE: *Empty declaration.*

$$\overline{B \vdash \quad \Rightarrow \{\}} \text{ in Env} \quad (59)$$

5.7.14 RULE: *Sequential declaration.*

$$\frac{B \vdash \text{strdec}_1 \Rightarrow E_1 \quad B \oplus E_1 \vdash \text{strdec}_2 \Rightarrow E_2}{B \vdash \text{strdec}_1 \langle ; \rangle \text{ strdec}_2 \Rightarrow E_1 + E_2} \quad (60)$$

Structure Bindings

$$\boxed{B \vdash \text{strbind} \Rightarrow SE}$$

5.7.15 RULE: *Structure binding*.

$$\frac{B \vdash \text{strex} \Rightarrow E \quad \langle B + \text{tynames } E \vdash \text{strbind} \Rightarrow SE \rangle}{B \vdash \text{strid} = \text{strex} \langle \text{and strbind} \rangle \Rightarrow \{ \text{strid} \mapsto E \} \langle + SE \rangle} \quad (61)$$

Signature Expressions

$$\boxed{B \vdash \text{sigexp} \Rightarrow E}$$

5.7.16 RULE: *Encapsulation signature expression*.

$$\frac{B \vdash \text{spec} \Rightarrow E}{B \vdash \text{sig spec end} \Rightarrow E} \quad (62)$$

5.7.17 RULE: *Signature identifier*.

$$\frac{B(\text{sigid}) = (T)E \quad T \cap (T \text{ of } B) = \emptyset}{B \vdash \text{sigid} \Rightarrow E} \quad (63)$$

The bound names of $B(\text{sigid})$ can always be renamed to satisfy $T \cap (T \text{ of } B) = \emptyset$, if necessary.

5.7.18 RULE: “Where type”.

Note this rule has $3 + 2 + 3 = 8$ premises.

$$\frac{\begin{array}{l} B \vdash \text{sigexp} \Rightarrow E \quad \text{tyvarseq} = \alpha^{(k)} \quad C \text{ of } B \vdash \text{ty} \Rightarrow \tau \\ E(\text{longtycon}) = (t, VE) \quad t \notin T \text{ of } B \\ \varphi = \{t \mapsto \Lambda \alpha^{(k)}. \tau\} \quad \Lambda \alpha^{(k)}. \tau \text{ admits equality, if } t \text{ does} \quad \varphi(E) \text{ well-formed} \end{array}}{B \vdash \text{sigexp where type tyvarseq longtycon} = \text{ty} \Rightarrow \varphi(E)} \quad (64)$$

$$\boxed{B \vdash \text{sigexp} \Rightarrow \Sigma}$$



The Definition has apparently introduced a new subsection consisting of a single rule, but has not named the subsection. This was particularly confusing to me until I read the L^AT_EX source for the Definition.

5.7.19 RULE: *Topmost signature expression*.

$$\frac{B \vdash \text{sigexp} \Rightarrow E \quad T = \text{tynames } E \setminus (T \text{ of } B)}{B \vdash \text{sigexp} \Rightarrow (T)E} \quad (65)$$

A signature expression *sigexp* which is an immediate constituent of a signature binding, a signature constraint, or a functor binding is elaborated to a signature, see rules 52, 53, 67, and 86.

Signature Declarations

$$B \vdash \text{sigdec} \Rightarrow G$$

5.7.20 RULE: *Single signature declaration.*

$$\frac{B \vdash \text{sigbind} \Rightarrow G}{B \vdash \text{signature sigbind} \Rightarrow G} \quad (66)$$

Signature Bindings

$$B \vdash \text{sigbind} \Rightarrow G$$

5.7.21 RULE: *Signature binding.*

$$\frac{B \vdash \text{sigexp} \Rightarrow \Sigma \quad \langle B \vdash \text{sigbind} \Rightarrow G \rangle}{B \vdash \text{sigid} = \text{sigexp} \langle \text{and sigbind} \rangle \Rightarrow \{ \text{sigid} \mapsto \Sigma \} \langle + G \rangle} \quad (67)$$

Specifications

$$B \vdash \text{spec} \Rightarrow E$$

5.7.22 RULE: *Value specification.*

$$\frac{C \text{ of } B \vdash \text{valdesc} \Rightarrow VE}{B \vdash \text{val valdesc} \Rightarrow \text{Clos } VE \text{ in Env}} \quad (68)$$

VE is determined by B and valdesc .

5.7.23 RULE: *Type specification.*

$$\frac{C \text{ of } B \vdash \text{typdesc} \Rightarrow TE \quad \forall (t, VE) \in \text{Ran } TE, t \text{ does not admit equality}}{B \vdash \text{type typdesc} \Rightarrow TE \text{ in Env}} \quad (69)$$

The typenames in TE are new.

5.7.24 RULE: *Equality type specification.*

$$\frac{C \text{ of } B \vdash \text{typdesc} \Rightarrow TE \quad \forall (t, VE) \in \text{Ran } TE, t \text{ admits equality}}{B \vdash \text{eqtype typdesc} \Rightarrow TE \text{ in Env}} \quad (70)$$

The typenames in TE are new.

5.7.25 RULE: *Datatype specification.*

$$\frac{\begin{array}{l} \forall (t, VE') \in \text{Ran } TE, t \notin T \text{ of } B \\ C \text{ of } B \oplus TE \vdash \text{datdesc} \Rightarrow VE, TE \\ TE \text{ maximises equality} \end{array}}{B \vdash \text{datatype datdesc} \Rightarrow (VE, TE) \text{ in Env}} \quad (71)$$

The typenames in TE are new.

5.7.26 RULE: *Datatype replication specification.*

$$\frac{B(\text{longtycon}) = (\theta, VE) \quad TE = \{ \text{tycon} \mapsto (\theta, VE) \}}{B \vdash \text{datatype tycon} \text{ -- } \text{datatype longtycon} \Rightarrow (VE, TE) \text{ in Env}} \quad (72)$$

5.7.27 RULE: *Exception specification.*

$$\frac{C \text{ of } B \vdash \text{exdesc} \Rightarrow VE}{B \vdash \text{exception exdesc} \Rightarrow VE \text{ in Env}} \quad (73)$$

VE is determined by B and exdesc and contains monotypes only.

5.7.28 RULE: *Structure specification.*

$$\frac{B \vdash \text{strdesc} \Rightarrow SE}{B \vdash \text{structure strdesc} \Rightarrow SE \text{ in Env}} \quad (74)$$

5.7.29 RULE: *Include signature specification.*

$$\frac{B \vdash \text{sigexp} \Rightarrow E}{B \vdash \text{include sigexp} \Rightarrow E} \quad (75)$$

5.7.30 RULE: *Empty specification.*

$$\frac{}{B \vdash \Rightarrow \{\}} \text{ in Env} \quad (76)$$

5.7.31 RULE: *Sequential specification.*

$$\frac{B \vdash \text{spec}_1 \Rightarrow E_1 \quad B \oplus E_1 \vdash \text{spec}_2 \Rightarrow E_2 \quad \text{Dom}(E_1) \cap \text{Dom}(E_2) = \emptyset}{B \vdash \text{spec}_1 \langle ; \rangle \text{spec}_2 \Rightarrow E_1 + E_2} \quad (77)$$

Note that no sequential specification is allowed to specify the same identifier twice.

5.7.32 RULE: *Sharing type specification.*

Note this has $2 + 2 + 2 = 6$ premises.

$$\frac{\begin{array}{l} B \vdash \text{spec} \Rightarrow E \quad E(\text{longtycon}_i) = (t_i, VE_i), i = 1..n \\ t \in \{t_1, \dots, t_n\} \quad t \text{ admits equality, if some } t_i \text{ does} \\ \{t_1, \dots, t_n\} \cap T \text{ of } B = \emptyset \quad \varphi = \{t_1 \mapsto t, \dots, t_n \mapsto t\} \end{array}}{B \vdash \text{spec sharing type longtycon}_1 = \dots = \text{longtycon}_n \Rightarrow \varphi(E)} \quad (78)$$

Value Descriptions

$$\boxed{C \vdash \text{valdesc} \Rightarrow VE}$$

5.7.33 RULE: *Value description.*

$$\frac{C \vdash \text{ty} \Rightarrow \tau \quad \langle C \vdash \text{valdesc} \Rightarrow VE \rangle}{C \vdash \text{vid} : \text{ty} \langle \text{and valdesc} \rangle \Rightarrow \{\text{vid} \mapsto (\tau, \mathbf{v})\} \langle + VE \rangle} \quad (79)$$

Type Descriptions

$$\boxed{C \vdash \text{typdesc} \Rightarrow TE}$$

5.7.34 RULE: *Type description.*

$$\frac{\begin{array}{l} \langle C \vdash \text{typdesc} \Rightarrow TE \quad t \notin \text{tynames } TE \rangle \\ \text{tyvarseq} = \alpha^{(k)} \quad t \notin T \text{ of } C \quad \text{arity } t = k \end{array}}{C \vdash \text{tyvarseq tycon} \langle \text{and typdesc} \rangle \Rightarrow \{\text{tycon} \mapsto (t, \{\})\} \langle + TE \rangle} \quad (80)$$

Note that the value environment in the resulting type structure must be empty. For example, `datatype s=C type t sharing type t=s` is a legal specification, but the type structure bound to `t` does not bind any value constructors.

Datatype Descriptions

$$C \vdash \text{datdesc} \Rightarrow VE, TE$$

5.7.35 RULE: *Datatype description.*

There are $2 + 3 = 5$ premises to this rule.

$$\frac{\langle C \vdash \text{datdesc}' \Rightarrow VE', TE' \quad \forall (t', VE'') \in \text{Ran } TE', t \neq t' \rangle}{\begin{array}{l} \text{tyvarseq} = \alpha^{(k)} \quad C, \alpha^{(k)} t \vdash \text{condesc} \Rightarrow VE \quad \text{arity } t = k \\ C \vdash \text{tyvarseq } \text{tycon} = \text{condesc} \langle \text{and } \text{datdesc}' \rangle \Rightarrow \\ \text{Clos } VE \langle + VE' \rangle, \{ \text{tycon} \mapsto (t, \text{Clos } VE) \} \langle + TE' \rangle \end{array}} \quad (81)$$

Constructor Descriptions

$$C, \tau \vdash \text{condesc} \Rightarrow VE$$

5.7.36 RULE:

$$\frac{\langle C \vdash \text{ty} \Rightarrow \tau' \rangle \quad \langle \langle C, \tau \vdash \text{condesc} \Rightarrow VE \rangle \rangle}{C, \tau \vdash \text{vid} \langle \text{of } \text{ty} \rangle \langle \langle \mid \text{condesc} \rangle \rangle \Rightarrow \{ \text{vid} \mapsto (\tau, c) \} \langle + \{ \text{vid} \mapsto (\tau' \rightarrow \tau, c) \} \rangle \langle \langle + VE \rangle \rangle} \quad (82)$$

Exception Descriptions

$$C \vdash \text{exdesc} \Rightarrow VE$$

5.7.37 RULE: *Exception description.*

$$\frac{\langle C \vdash \text{ty} \Rightarrow \tau \quad \text{tyvars}(\tau) = \emptyset \rangle \quad \langle \langle C \vdash \text{exdesc} \Rightarrow VE \rangle \rangle}{C \vdash \text{vid} \langle \text{of } \text{ty} \rangle \langle \langle \text{and } \text{exdesc} \rangle \rangle \Rightarrow \{ \text{vid} \mapsto (\text{exn}, e) \} \langle + \{ \text{vid} \mapsto (\tau \rightarrow \text{exn}, e) \} \rangle \langle \langle + VE \rangle \rangle} \quad (83)$$

Structure Descriptions

$$B \vdash \text{strdesc} \Rightarrow SE$$

5.7.38 RULE: *Structure description.*

$$\frac{B \vdash \text{sigexp} \Rightarrow E \quad \langle B + \text{tynames } E \vdash \text{strdesc} \Rightarrow SE \rangle}{B \vdash \text{strid} : \text{sigexp} \langle \text{and } \text{strdesc} \rangle \Rightarrow \{ \text{strid} \mapsto E \} \langle + SE \rangle} \quad (84)$$

Functor Declarations

$$B \vdash \text{fundec} \Rightarrow F$$

5.7.39 RULE: *Single functor declaration.*

$$\frac{B \vdash \text{funbind} \Rightarrow F}{B \vdash \text{functor } \text{funbind} \Rightarrow F} \quad (85)$$

Functor Bindings

$$B \vdash \text{funbind} \Rightarrow F$$

5.7.40 RULE: *Functor binding.*

$$\frac{\begin{array}{l} B \vdash \text{sigexp} \Rightarrow (T)E \quad B \oplus \{ \text{strid} \mapsto E \} \vdash \text{strex} \Rightarrow E' \\ T \cap (T \text{ of } B) = \emptyset \quad T' = \text{tynames } E' \setminus ((T \text{ of } B) \cup T) \\ \langle B \vdash \text{funbind} \Rightarrow F \rangle \end{array}}{B \vdash \text{funid} (\text{strid} : \text{sigexp}) = \text{strex} \langle \text{and } \text{funbind} \rangle \Rightarrow \{ \text{funid} \mapsto (T)(E, (T')E') \} \langle \langle + F \rangle \rangle} \quad (86)$$

Since $\oplus$ is used, any type name t in E acts like a constant in the functor body; in particular, it ensures that further names generated during elaboration of the body are distinct from t . The set T' is chosen such that every name free in $(T)E$ or $(T)(E, (T')E')$ is free in B .

Top-level Declarations

$$\boxed{B \vdash \text{topdec} \Rightarrow B'}$$

5.7.41 RULE: *Structure-level declaration.*

$$\frac{\begin{array}{l} B \vdash \text{strdec} \Rightarrow E \quad \langle B \oplus E \vdash \text{topdec} \Rightarrow B' \rangle \\ B'' = (\text{tynames } E, E) \text{ in Basis } \langle +B' \rangle \quad \text{tyvars } B'' = \emptyset \end{array}}{B \vdash \text{strdec} \quad \langle \text{topdec} \rangle \Rightarrow B''} \quad (87)$$

No free type variables enter the basis: if $B \vdash \text{topdec} \Rightarrow B'$ then $\text{tyvars}(B') = \emptyset$.

5.7.42 RULE: *Signature Declaration.*

$$\frac{\begin{array}{l} B \vdash \text{sigdec} \Rightarrow G \quad \langle B \oplus G \vdash \text{topdec} \Rightarrow B' \rangle \\ B'' = (\text{tynames } G, G) \text{ in Basis } \langle +B' \rangle \end{array}}{B \vdash \text{sigdec} \quad \langle \text{topdec} \rangle \Rightarrow B''} \quad (88)$$

No free type variables enter the basis: if $B \vdash \text{topdec} \Rightarrow B'$ then $\text{tyvars}(B') = \emptyset$.

5.7.43 RULE: *Functor declaration.*

$$\frac{\begin{array}{l} B \vdash \text{fundec} \Rightarrow F \quad \langle B \oplus F \vdash \text{topdec} \Rightarrow B' \rangle \\ B'' = (\text{tynames } F, F) \text{ in Basis } \langle +B' \rangle \quad \text{tyvars } B'' = \emptyset \end{array}}{B \vdash \text{fundec} \quad \langle \text{topdec} \rangle \Rightarrow B''} \quad (89)$$

No free type variables enter the basis: if $B \vdash \text{topdec} \Rightarrow B'$ then $\text{tyvars}(B') = \emptyset$.

DYNAMIC SEMANTICS FOR THE CORE

6.1. REDUCED SYNTAX

6.1.1 CLAUSE: *Core syntax reduced.*

Since types are mostly dealt with in the static semantics, the Core syntax is reduced by the following transformations, for the purpose of the dynamic semantics:

- (1) All explicit type ascriptions “: *ty*” are omitted, and qualifications “of *ty*” are omitted from constructor and exception bindings.
- (2) The Core phrase classes Ty and TyRow are omitted.

6.2. SIMPLE OBJECTS

6.2.1 DEFINITION: *Classes of simple objects.*

All objects in the dynamic semantics are built from the identifier classes together with the simple object classes listed below (note that basic values are discussed in their own section):

a	$\in$	Addr	addresses
en	$\in$	ExName	exception names
b	$\in$	BasVal	basic values
sv	$\in$	SVal	special values
		{FAIL}	failure

6.2.2 CLAUSE: *Infinite set of addresses.*

Addr forms an infinite set.

6.2.3 CLAUSE: *Infinite set of exception names.*

ExName forms an infinite set.

6.2.4 CLAUSE: *Special values are values of special constants.*

SVal is the class of values denoted by the special constants SCon. Each integer, word, or real constant denotes a value according to normal mathematical conventions; each string or character constant denotes a sequence of characters as explained in Section 2.2.

6.2.5 DEFINITION: *Value of a special constant.*

The value denoted by *scon* is written $\text{val}(\textcolor{red}{scon})$.

6.2.6 CLAUSE: *Failure “value”.*

FAIL is the result of a failing attempt to match a value and a pattern. Thus FAIL is neither a value nor an exception, but simply a semantic object used in the rules to express operationally how matching proceeds.

6.2.7 CLAUSE: *Exception constructors evaluate to names.*

Exception constructors evaluate to exception names. This accomodates the generative nature of exception bindings: each evaluation of a declaration of an exception constructor binds it to a new unique name.

6.3. COMPOUND OBJECTS

6.3.1 DEFINITION: *Compound objects.*

The compound objects for dynamic semantics are listed thus:

$$\begin{aligned}
 v &\in \text{Val} = \{:=\} \cup \text{SVal} \cup \text{BasVal} \cup \text{VId} \\
 &\quad \cup (\text{VId} \times \text{Val}) \cup \text{ExVal} \\
 &\quad \cup \text{Record} \cup \text{Addr} \cup \text{FcnClosure} \\
 r &\in \text{Record} = \text{Lab} \xrightarrow{\text{fin}} \text{Val} \\
 e &\in \text{ExVal} = \text{ExName} \cup (\text{ExName} \times \text{Val}) \\
 [e] \text{ or } p &\in \text{Pack} = \text{ExVal} \\
 (\text{match}, E, VE) &\in \text{FcnClosure} = \text{Match} \times \text{Env} \times \text{ValEnv} \\
 mem &\in \text{Mem} = \text{Addr} \xrightarrow{\text{fin}} \text{Val} \\
 ens &\in \text{ExNameSet} = \text{Fin}(\text{ExName}) \\
 (mem, ens) \text{ or } s &\in \text{State} = \text{Mem} \times \text{ExNameSet} \\
 (SE, TE, VE \text{ or } E) &\in \text{Env} = \text{StrEnv} \times \text{TyEnv} \times \text{ValEnv} \\
 SE &\in \text{StrEnv} = \text{StrId} \xrightarrow{\text{fin}} \text{Env} \\
 TE &\in \text{TyEnv} = \text{TyCon} \xrightarrow{\text{fin}} \text{ValEnv} \\
 VE &\in \text{ValEnv} = \text{VId} \xrightarrow{\text{fin}} \text{Val} \times \text{IdStatus}
 \end{aligned}$$

6.3.2 CONVENTION: *Projection, injection, modification.*

The notions of projection, injection, and modification are adapted from the static semantics.

6.3.3 CLAUSE: *Object classes disjoint.*

The object classes thus defined are all disjoint, so there is no ambiguity in taking unions as disjoint unions.

6.3.4 CLAUSE: *Exception values and packets.*

Observe that ExVal and Pack (exception values and packets) are isomorphic classes, but the latter class corresponds to exceptions which have been raised, and therefore has different semantic significance from the former, which is just a subclass of values.

6.3.5 COMMENT: *Caution about overloaded notation.*

Although the same names (e.g., E for an environment) are used as in the static semantics, the objects denoted are different. This need cause no confusion since the static and dynamic semantics are presented separately.

6.4. BASIC VALUES

6.4.1 CLAUSE: *Simplified set of basic values.*

The Definition takes BasVal to be the singleton set $\{=\}$, but allows for a larger set of basic values defined by libraries.

6.4.2 CLAUSE: *APPLY gives meaning to basic values.*

The semantical meaning of basic values is represented by a function

$$\text{APPLY} : \text{BasVal} \times \text{Val} \rightarrow \text{Val} \cup \text{Pack}$$

which satisfies $\text{APPLY}(=, \{1 \mapsto v_1, 2 \mapsto v_2\})$ is **true** if v_1 and v_2 are identical values, and **false** otherwise.

6.4.3 COMMENT: *Problems with APPLY.*

As Rossberg [Ros18b, §6] and Kahrs [Kah93, §§11.1–3] note, the APPLY function has no access to program state. This implies the library primitives may not be stateful, and moreover that a lot of interesting primitives (e.g., arrays,

I/O streams, etc.) could not be added to the language without extending the Definition itself.

But Rossberg observes, any nontrivial library type requires an extension of the Definition of values or state anyways (and equality types — for, e.g., `array`). The Definition should have contained a comment regarding this.

6.5. BASIC EXCEPTIONS

6.5.1 CLAUSE: *Initial basis.*

A subset $\text{BasExName} \subset \text{ExName}$ of the exception names are bound to predefined exception constructors in the initial dynamic basis. These names are denoted by the identifiers to which they are bound in the initial basis, and are as follows:

Match **Bind**

These exceptions are raised upon failure of pattern-matching in evaluating a function or a *valbind*, as detailed in the rules that follow.

6.5.2 CLAUSE: *Match raised by nonexhaustive patterns.*

The exception **Match** can only be raised for a match which is not exhaustive, and has therefore been flagged by the compiler (§4.11.4).

6.6. FUNCTION CLOSURE

6.6.1 COMMENT: *Closures are “function values”.*

Intuitively, a function closure is precisely the “value” for a function expression. This is critical when trying to use, e.g., equational reasoning for Standard ML.

6.6.2 CLAUSE: *Informal explanation of closures.*

A function closure (match, E, VE) may be informally described as follows: when the function closure is applied to a value v , *match* will be evaluated against v , in the environment E modified in a special sense by VE (to allow for recursive functions). This happens by evaluating *match* in $E + \text{Rec } VE$, which will be defined shortly.

6.6.3 DEFINITION: *Rec operator on value environments.*

The function

$$\text{Rec} : \text{ValEnv} \rightarrow \text{ValEnv}$$

is defined as follows:

- (1) $\text{Dom}(\text{Rec } VE) = \text{Dom } VE$
- (2) If $VE(\text{vid}) \notin \text{FcnClosure} \times \{\mathbf{v}\}$, then $(\text{Rec } VE)(\text{vid}) = VE(\text{vid})$
- (3) If $VE(\text{vid}) = ((\text{match}', E', VE'), \mathbf{v})$ then $(\text{Rec } VE)(\text{vid}) = ((\text{match}', E', VE'), \mathbf{v})$

6.6.4 COMMENT: *Rec VE “unrolls” function closures once.*

The effect of this definition is that — before the application of the closure (match, E, VE) to v — the function closures in $\text{Ran } VE$ [not a type: $\text{Ran } VE$ is correct, see next example] are “unrolled” once, to prepare for their possible recursive application during the evaluation of *match* upon v .

This ensures all semantic objects are finite (by controlling the unrolling of recursion).

6.6.5 EXAMPLE: *Example unrolling recursive function calls.*

It makes more sense to see this rule in action. I’m adapting the example from the Commentary [MT91, pp.1–12]. Consider the example code:

```

datatype NAT = Zero | Succ of NAT;

fun twice(Zero) = Zero
  | twice(Succ x) = Succ(Succ(twice(x)));

twice(Zero);

```

The desugared code “`twice = fn match`” will evaluate to

$$VE = \{\text{twice} \mapsto (\text{match}, E_1, \{\})\}.$$

The problem is that, looking at the VE appearing in the closure bound to `twice`: it’s precisely $VE_{\text{twice}} = \{\}$! So trying to lookup the identifier `twice` for a recursive function call will result in a failure.

However, we don’t look up `twice` in $VE_{\text{twice}} = \{\}$ but instead we’re using

$$\text{Rec } VE = \{\text{twice} \mapsto (\text{match}, E_1, VE)\}.$$

This allows for proper recursive function calls.

6.6.6 COMMENT: *Used only twice.*

The `Rec` operator is used only twice: in Rules (102) [the rule for evaluating an application expression “`exp atexp`”], and (126) [governing recursive value bindings of the form “`rec valbind`”].

6.7. INFERENCE RULES

6.7.1 DEFINITION: *Judgement form.*

The semantic rules govern judgemental forms like

$$s, A \vdash \text{phrase} \Rightarrow A', s'$$

where A is usually an environment, A' is some semantic object, and here s, s' are states before and after the evaluation which occurs in the judgement.

6.7.2 DEFINITION: *Side-conditions.*

Some hypotheses in the rules are not of the form we just described. They are called “**Side-Conditions**”.

6.7.3 CLAUSE: *Convention for options.*

The convention for options is the same as for the Core static semantics.

6.7.4 CONVENTION: *State convention.*

In most cases, the state can be omitted from the inference rule. When omitted, the conventions for restoring them as is follows: If the rule is presented in the form

$$\frac{A_1 \vdash \text{phrase}_1 \Rightarrow A'_1 \quad A_2 \vdash \text{phrase}_2 \Rightarrow A'_2 \quad \cdots \quad \cdots \quad A_n \vdash \text{phrase}_n \Rightarrow A'_n}{A \vdash \text{phrase} \Rightarrow A'}$$

then the full form is intended to be

$$\frac{s_0, A_1 \vdash \text{phrase}_1 \Rightarrow A'_1, s_1 \quad s_1, A_2 \vdash \text{phrase}_2 \Rightarrow A'_2, s_2 \quad \cdots \quad \cdots \quad s_{n-1}, A_n \vdash \text{phrase}_n \Rightarrow A'_n, s_n}{s_0, A \vdash \text{phrase} \Rightarrow A', s_n}$$

(Any side-conditions are left unaltered).

Thus the left-to-right order of the hypotheses indicates the order of evaluation. Note that in the case $n = 0$, when there are no hypotheses (except possibly side-conditions), we have $s_n = s_0$; this implies that the rule causes no side effect.

The Definition calls this convention the “**State Convention**”, and it must be applied to each version of a rule obtained by inclusion or omission of its options.

[Used in: 7.3.4]

6.7.5 CONVENTION: *Exception convention.*

The “**Exception Convention**” is adopted to deal with the propagation of exception packets p . For each rule whose full form (ignoring side-conditions) is

$$\frac{s_1, A_1 \vdash \text{phrase}_1 \Rightarrow A'_1, s'_1 \quad \cdots \quad s_n, A_n \vdash \text{phrase}_n \Rightarrow A'_n, s'_n}{s, A \vdash \text{phrase} \Rightarrow A', s'}$$

and for each k , $1 \leq k \leq n$, for which the result A'_k is not a packet p , an extra rule is added of the form

$$\frac{s_1, A_1 \vdash \text{phrase}_1 \Rightarrow A'_1, s'_1 \quad \cdots \quad s_k, A_k \vdash \text{phrase}_k \Rightarrow p', s'}{s, A \vdash \text{phrase} \Rightarrow p', s'}$$

where p' does not occur in the original rule. [The only exception is rule 104, describing how exception handlers work.] This indicates that evaluation of phrases in the hypothesis terminates with the first whose result is a packet (other than one already treated in the rule), and this packet is the result of the phrase in the conclusion.

[Used in: 7.3.4]

6.7.6 CONVENTION: *Compound variables.*

The last convention allows for “compound variables” (built from the metavariables and a slash “/”) to range over unions of semantic objects.

For instance the compound variable v/p ranges over $\text{Val} \cup \text{Pack}$. We also allow x/FAIL to range over $X \cup \{\text{FAIL}\}$ where x ranges over X ; furthermore, we extend environment modification to allow for failure as follows:

$$VE + \text{FAIL} = \text{FAIL}.$$

Atomic Expressions

$$E \vdash \text{atexp} \Rightarrow v/p$$

6.7.7 RULE: *Special constant.*

$$\overline{E \vdash \text{scon} \Rightarrow \text{val}(\text{scon})} \quad (90)$$

6.7.8 RULE: *Value variable.*

$$\frac{E(\text{longvid}) = (v, \text{is})}{E \vdash \text{longvid} \Rightarrow v} \quad (91)$$

As in the static semantics, value identifiers are looked up in the environment and the identifier status is not used.

6.7.9 RULE: *Record expression.*

$$\frac{\langle E \vdash \text{exprow} \Rightarrow r \rangle}{E \vdash \{ \langle \text{exprow} \rangle \} \Rightarrow \{ \} \langle + r \rangle \text{ in Val}} \quad (92)$$

6.7.10 RULE: *Let expressions.*

$$\frac{E \vdash \text{dec} \Rightarrow E' \quad E + E' \vdash \text{exp} \Rightarrow v}{E \vdash \text{let } \text{dec} \text{ in } \text{exp} \text{ end} \Rightarrow v} \quad (93)$$

6.7.11 RULE: *Parenthesised expression.*

$$\frac{E \vdash \text{exp} \Rightarrow v}{E \vdash (\text{exp}) \Rightarrow v} \quad (94)$$

Expression Rows

$$E \vdash \text{exprow} \Rightarrow r/p$$

6.7.12 RULE: *Labelled expressions.*

$$\frac{E \vdash \text{exp} \Rightarrow v \quad \langle E \vdash \text{exprow} \Rightarrow r \rangle}{E \vdash \text{lab} = \text{exp} \langle \text{ , exprow} \rangle \Rightarrow \{ \text{lab} \mapsto v \} \langle + r \rangle} \quad (95)$$

We may think of components as being evaluated from left to right, because of the state and exception conventions.

Expressions

$$E \vdash \text{exp} \Rightarrow v/p$$

6.7.13 RULE: *Atomic expressions.*

$$\frac{E \vdash \text{atexp} \Rightarrow v}{E \vdash \text{atexp} \Rightarrow v} \quad (96)$$

6.7.14 RULE: *Constructor application.*

$$\frac{E \vdash \text{exp} \Rightarrow \text{vid} \quad \text{vid} \neq \mathbf{ref} \quad E \vdash \text{atexp} \Rightarrow v}{E \vdash \text{exp atexp} \Rightarrow (\text{vid}, v)} \quad (97)$$

Note that none of the rules for function application has a premise in which the operator evaluates to a constructed value, a record or an address. This is because we are interested in the evaluation of well-typed programs only, and in such programs *exp* will always have a functional type.

6.7.15 RULE: *Exception constructor application.*

$$\frac{E \vdash \text{exp} \Rightarrow \text{en} \quad E \vdash \text{atexp} \Rightarrow v}{E \vdash \text{exp atexp} \Rightarrow (\text{en}, v)} \quad (98)$$

Note that none of the rules for function application has a premise in which the operator evaluates to a constructed value, a record or an address. This is because we are interested in the evaluation of well-typed programs only, and in such programs *exp* will always have a functional type.

6.7.16 RULE: *Reference application.*

$$\frac{s, E \vdash \text{exp} \Rightarrow \mathbf{ref} \text{ , } s' \quad s', E \vdash \text{atexp} \Rightarrow v, s'' \quad a \notin \text{Dom}(\text{mem of } s'')}{s, E \vdash \text{exp atexp} \Rightarrow a, s'' + \{a \mapsto v\}} \quad (99)$$

- (1) The side condition ensures that a new address is chosen. There are no rules concerning disposal of inaccessible addresses.
- (2) Note that none of the rules for function application has a premise in which the operator evaluates to a constructed value, a record or an address. This is because we are interested in the evaluation of well-typed programs only, and in such programs *exp* will always have a functional type.

6.7.17 RULE: *Assignment application.*

$$\frac{s, E \vdash \text{exp} \Rightarrow := \text{ , } s' \quad s', E \vdash \text{atexp} \Rightarrow \{1 \mapsto a, 2 \mapsto v\}, s''}{s, E \vdash \text{exp atexp} \Rightarrow \{ \} \text{ in Val, } s'' + \{a \mapsto v\}} \quad (100)$$

Note that none of the rules for function application has a premise in which the operator evaluates to a constructed value, a record or an address. This is because we are interested in the evaluation of well-typed programs only, and in such programs *exp* will always have a functional type.

6.7.18 RULE: *Basic function application.*

$$\frac{E \vdash \text{exp} \Rightarrow b \quad E \vdash \text{atexp} \Rightarrow v \quad \text{APPLY}(b, v) = v'/p}{E \vdash \text{exp atexp} \Rightarrow v'/p} \quad (101)$$

Note that none of the rules for function application has a premise in which the operator evaluates to a constructed value, a record or an address. This is because we are interested in the evaluation of well-typed programs only, and in such programs *exp* will always have a functional type.

6.7.19 RULE: *Closure application.*

$$\frac{E \vdash \text{exp} \Rightarrow (\text{match}, E', VE) \quad E \vdash \text{atexp} \Rightarrow v \quad E' + \text{Rec } VE, v \vdash \text{match} \Rightarrow v'}{E \vdash \text{exp atexp} \Rightarrow v'} \quad (102)$$

Note that none of the rules for function application has a premise in which the operator evaluates to a constructed value, a record or an address. This is because we are interested in the evaluation of well-typed programs only, and in such programs *exp* will always have a functional type.

6.7.20 RULE: *Failing closure application.*

$$\frac{E \vdash \text{exp} \Rightarrow (\text{match}, E', VE) \quad E \vdash \text{atexp} \Rightarrow v \quad E' + \text{Rec } VE, v \vdash \text{match} \Rightarrow \text{FAIL}}{E \vdash \text{exp atexp} \Rightarrow [\text{Match}]} \quad (103)$$

Note that none of the rules for function application has a premise in which the operator evaluates to a constructed value, a record or an address. This is because we are interested in the evaluation of well-typed programs only, and in such programs *exp* will always have a functional type.

6.7.21 RULE: *Unexceptional handler.*

$$\frac{E \vdash \text{exp} \Rightarrow v}{E \vdash \text{exp handle match} \Rightarrow v} \quad (104)$$

This is the only rule to which the exception convention does not apply. If the operator evaluates to a packet then rule 105 or rule 106 must be used.

6.7.22 RULE: *Handled exception.*

$$\frac{E \vdash \text{exp} \Rightarrow [e] \quad E, e \vdash \text{match} \Rightarrow v}{E \vdash \text{exp handle match} \Rightarrow v} \quad (105)$$

6.7.23 RULE: *Unhandled exception.*

$$\frac{E \vdash \text{exp} \Rightarrow [e] \quad E, e \vdash \text{match} \Rightarrow \text{FAIL}}{E \vdash \text{exp handle match} \Rightarrow [e]} \quad (106)$$

Packets that are not handled by the *match* propagate.

6.7.24 RULE: *Raise exception.*

$$\frac{E \vdash \text{exp} \Rightarrow e}{E \vdash \text{raise exp} \Rightarrow [e]} \quad (107)$$

6.7.25 RULE: *Function.*

This evaluates a function expression to a function value.

$$\overline{E \vdash \text{fn match} \Rightarrow (\text{match}, E, \{\})} \quad (108)$$

The third component of the function closure is empty because the match does not introduce new recursively defined values.

Matches

$$E, v \vdash \text{match} \Rightarrow v'/p/\text{FAIL}$$

6.7.26 RULE: *Match 1.*

This intuitively formalizes “Go with the first match possible” semantics.

$$\frac{E, v \vdash \text{mrule} \Rightarrow v'}{E, v \vdash \text{mrule} \langle \mid \text{match} \rangle \Rightarrow v'} \quad (109)$$

6.7.27 RULE: *Match 2.*

$$\frac{E, v \vdash \text{mrule} \Rightarrow \text{FAIL}}{E, v \vdash \text{mrule} \Rightarrow \text{FAIL}} \quad (110)$$

6.7.28 RULE: *Match 3.*

Given a leading *mrule* which fails to match, but there is a rule in the rest of the *match* which either matches or fails... we go with that instead.

$$\frac{E, v \vdash \text{mrule} \Rightarrow \text{FAIL} \quad E, v \vdash \text{match} \Rightarrow v'/\text{FAIL}}{E, v \vdash \text{mrule} \mid \text{match} \Rightarrow v'/\text{FAIL}} \quad (111)$$

Comment: A value v occurs on the left of the turnstile, in evaluating a *match*. We may think of a *match* as being evaluated *against* a value; similarly, we may think of a pattern as being evaluated *against* a value. Alternative match rules are tried from left to right.

Match Rules

$$E, v \vdash \text{mrule} \Rightarrow v'/p/\text{FAIL}$$

6.7.29 RULE: *Match a clause successfully.*

$$\frac{E, v \vdash \text{pat} \Rightarrow VE \quad E + VE \vdash \text{exp} \Rightarrow v'}{E, v \vdash \text{pat} \Rightarrow \text{exp} \Rightarrow v'} \quad (112)$$

6.7.30 RULE: *Failing to match a pattern.*

$$\frac{E, v \vdash \text{pat} \Rightarrow \text{FAIL}}{E, v \vdash \text{pat} \Rightarrow \text{exp} \Rightarrow \text{FAIL}} \quad (113)$$

Declarations

$$E \vdash \text{dec} \Rightarrow E'/p$$

6.7.31 RULE: *Value declaration.*

$$\frac{E \vdash \text{valbind} \Rightarrow VE}{E \vdash \text{val tyvarseq valbind} \Rightarrow VE \text{ in Env}} \quad (114)$$

6.7.32 RULE: *Type alias.*

$$\frac{\vdash \text{typbind} \Rightarrow TE}{E \vdash \text{type typbind} \Rightarrow TE \text{ in Env}} \quad (115)$$

6.7.33 RULE: *Datatype declaration.*

$$\frac{\vdash \text{datbind} \Rightarrow VE, TE}{E \vdash \text{datatype datbind} \Rightarrow (VE, TE) \text{ in Env}} \quad (116)$$

6.7.34 RULE: *Datatype replication.*

$$\frac{E(\text{longtycon}) = VE}{E \vdash \text{datatype tycon} \text{ --- datatype longtycon} \Rightarrow (VE, \{\text{tycon} \mapsto VE\}) \text{ in Env}} \quad (117)$$

6.7.35 RULE: *Abstract typing declaration.*

$$\frac{\vdash \text{datbind} \Rightarrow VE, TE \quad E + VE \vdash \text{dec} \Rightarrow E'}{E \vdash \text{abstype datbind with dec end} \Rightarrow E'} \quad (118)$$

6.7.36 RULE: *Exception declaration.*

$$\frac{E \vdash \text{exbind} \Rightarrow VE}{E \vdash \text{exception exbind} \Rightarrow VE \text{ in Env}} \quad (119)$$

6.7.37 RULE: *Local declaration.*

$$\frac{E \vdash \text{dec}_1 \Rightarrow E_1 \quad E + E_1 \vdash \text{dec}_2 \Rightarrow E_2}{E \vdash \text{local dec}_1 \text{ in dec}_2 \text{ end} \Rightarrow E_2} \quad (120)$$

6.7.38 RULE: *Open declaration.*

$$\frac{E(\text{longstrid}_1) = E_1 \quad \cdots \quad E(\text{longstrid}_n) = E_n}{E \vdash \text{open longstrid}_1 \cdots \text{longstrid}_n \Rightarrow E_1 + \cdots + E_n} \quad (121)$$

6.7.39 RULE: *Empty declaration.*

$$\frac{}{E \vdash \Rightarrow \{\} \text{ in Env}} \quad (122)$$

6.7.40 RULE: *Sequential declaration.*

$$\frac{E \vdash \text{dec}_1 \Rightarrow E_1 \quad E + E_1 \vdash \text{dec}_2 \Rightarrow E_2}{E \vdash \text{dec}_1 \langle ; \rangle \text{dec}_2 \Rightarrow E_1 + E_2} \quad (123)$$

Value Bindings

$$\boxed{E \vdash \text{valbind} \Rightarrow VE/p}$$

6.7.41 RULE: *Nonrecursive, possibly simultaneous, bindings.*

$$\frac{E \vdash \text{exp} \Rightarrow v \quad E, v \vdash \text{pat} \Rightarrow VE \quad \langle E \vdash \text{valbind} \Rightarrow VE' \rangle}{E \vdash \text{pat} = \text{exp} \langle \text{and valbind} \rangle \Rightarrow VE \langle + VE' \rangle} \quad (124)$$

6.7.42 RULE: *Binding exception during declaration.*

$$\frac{E \vdash \text{exp} \Rightarrow v \quad E, v \vdash \text{pat} \Rightarrow \text{FAIL}}{E \vdash \text{pat} = \text{exp} \langle \text{and valbind} \rangle \Rightarrow [\text{Bind}]} \quad (125)$$

6.7.43 RULE: *Recursive value binding.*

$$\frac{E \vdash \text{valbind} \Rightarrow VE}{E \vdash \text{rec valbind} \Rightarrow \text{Rec } VE} \quad (126)$$

Type Bindings

$$\boxed{\vdash \text{typbind} \Rightarrow TE}$$

6.7.44 RULE: *Type binding.*

$$\frac{\langle \vdash \text{typbind} \Rightarrow TE \rangle}{\vdash \text{tyvarseq tycon} = \text{ty} \langle \text{and typbind} \rangle \Rightarrow \{\text{tycon} \mapsto \{\}\} \langle + TE \rangle} \quad (127)$$

Datatype Bindings

$$\boxed{\vdash \text{datbind} \Rightarrow VE, TE}$$

6.7.45 RULE: *Datatype binding.*

$$\frac{\vdash \text{conbind} \Rightarrow VE \quad \langle \vdash \text{datbind}' \Rightarrow VE', TE' \rangle}{\vdash \text{tyvarseq tycon} = \text{conbind} \langle \text{and datbind}' \rangle \Rightarrow VE \langle + VE' \rangle, \{ \text{tycon} \mapsto VE \} \langle + TE' \rangle} \quad (128)$$

Constructor Bindings

$$\boxed{\vdash \text{conbind} \Rightarrow VE}$$

6.7.46 RULE: *Constructor binding.*

$$\frac{\langle \vdash \text{conbind} \Rightarrow VE \rangle}{\vdash \text{vid} \langle | \text{conbind} \rangle \Rightarrow \{ \text{vid} \mapsto (\text{vid}, c) \} \langle + VE \rangle} \quad (129)$$

Exception Bindings

$$\boxed{E \vdash \text{exbind} \Rightarrow VE}$$

6.7.47 RULE: *Exception binding.*

$$\frac{\text{en} \notin \text{ens of } s \quad s' = s + \{ \text{en} \} \quad \langle s', E \vdash \text{exbind} \Rightarrow VE, s'' \rangle}{s, E \vdash \text{vid} \langle \text{and exbind} \rangle \Rightarrow \{ \text{vid} \mapsto (\text{en}, e) \} \langle + VE \rangle, s' \langle '' \rangle} \quad (130)$$

The two side conditions ensure that a new exception name is generated and recorded as “used” in subsequent states.

6.7.48 RULE: *Looking up exception binding in environment.*

When a *longvid* is bound to an exception.

$$\frac{E(\text{longvid}) = (\text{en}, e) \quad \langle E \vdash \text{exbind} \Rightarrow VE \rangle}{E \vdash \text{vid} = \text{longvid} \langle \text{and exbind} \rangle \Rightarrow \{ \text{vid} \mapsto (\text{en}, e) \} \langle + VE \rangle} \quad (131)$$

Atomic Patterns

$$\boxed{E, v \vdash \text{atpat} \Rightarrow VE/\text{FAIL}}$$

6.7.49 RULE: *Wildcard pattern.*

$$\overline{E, v \vdash _ \Rightarrow \{ \}} \quad (132)$$

6.7.50 RULE: *Successfully match against special constant.*

$$\frac{v = \text{val}(\text{scon})}{E, v \vdash \text{scon} \Rightarrow \{ \}} \quad (133)$$

6.7.51 RULE: *Fail to match against special constant.*

$$\frac{v \neq \text{val}(\text{scon})}{E, v \vdash \text{scon} \Rightarrow \text{FAIL}} \quad (134)$$

Any evaluation resulting in FAIL must do so because rule 134, rule 137, rule 145, or rule 147 has been applied.

6.7.52 RULE: *Variable pattern.*

$$\frac{\text{vid} \notin \text{Dom}(E) \text{ or is of } E(\text{vid}) = v}{E, v \vdash \text{vid} \Rightarrow \{ \text{vid} \mapsto (v, v) \}} \quad (135)$$

6.7.53 RULE: *Value/exception constructor pattern.*

$$\frac{E(\text{longvid}) = (v, \text{is}) \quad \text{is} \neq v}{E, v \vdash \text{longvid} \Rightarrow \{ \}} \quad (136)$$

6.7.54 RULE: *Value/exception constructor pattern fail.*

$$\frac{E(\text{longvid}) = (v', is) \quad is \neq \mathbf{v} \quad v \neq v'}{E, v \vdash \text{longvid} \Rightarrow \text{FAIL}} \quad (137)$$

Any evaluation resulting in FAIL must do so because rule 134, rule 137, rule 145, or rule 147 has been applied.

6.7.55 RULE: *Record pattern.*

$$\frac{v = \{\} \langle +r \rangle \text{ in Val} \quad \langle E, r \vdash \text{patrow} \Rightarrow VE/\text{FAIL} \rangle}{E, v \vdash \{ \langle \text{patrow} \rangle \} \Rightarrow \{\} \langle + VE/\text{FAIL} \rangle} \quad (138)$$

6.7.56 RULE: *Parenthesised pattern.*

$$\frac{E, v \vdash \text{pat} \Rightarrow VE/\text{FAIL}}{E, v \vdash (\text{pat}) \Rightarrow VE/\text{FAIL}} \quad (139)$$

Pattern Rows

$$\boxed{E, r \vdash \text{patrow} \Rightarrow VE/\text{FAIL}}$$

6.7.57 RULE: *Wildcard record.*

$$\overline{E, r \vdash \dots \Rightarrow \{\}} \quad (140)$$

6.7.58 RULE: *Record component with inherited FAIL.*

$$\frac{E, r(\text{lab}) \vdash \text{pat} \Rightarrow \text{FAIL}}{E, r \vdash \text{lab} = \text{pat} \langle \text{ , patrow} \rangle \Rightarrow \text{FAIL}} \quad (141)$$

For well-typed programs *lab* will be in the domain of *r*.

6.7.59 RULE: *Record component.*

$$\frac{E, r(\text{lab}) \vdash \text{pat} \Rightarrow VE \quad \langle E, r \vdash \text{patrow} \Rightarrow VE'/\text{FAIL} \rangle}{E, r \vdash \text{lab} = \text{pat} \langle \text{ , patrow} \rangle \Rightarrow VE \langle + VE'/\text{FAIL} \rangle} \quad (142)$$

For well-typed programs *lab* will be in the domain of *r*.

Patterns

$$\boxed{E, v \vdash \text{pat} \Rightarrow VE/\text{FAIL}}$$

6.7.60 RULE: *Atomic pattern.*

$$\frac{E, v \vdash \text{atpat} \Rightarrow VE/\text{FAIL}}{E, v \vdash \text{atpat} \Rightarrow VE/\text{FAIL}} \quad (143)$$

6.7.61 RULE: *Construction pattern.*

$$\frac{E(\text{longvid}) = (vid, \mathbf{c}) \quad vid \neq \mathbf{ref} \quad v = (vid, v') \quad E, v' \vdash \text{atpat} \Rightarrow VE/\text{FAIL}}{E, v \vdash \text{longvid atpat} \Rightarrow VE/\text{FAIL}} \quad (144)$$

6.7.62 RULE: *Construction pattern failure.*

$$\frac{E(\text{longvid}) = (vid, \mathbf{c}) \quad vid \neq \mathbf{ref} \quad v \notin \{vid\} \times \text{Val}}{E, v \vdash \text{longvid atpat} \Rightarrow \text{FAIL}} \quad (145)$$

Any evaluation resulting in FAIL must do so because rule 134, rule 137, rule 145, or rule 147 has been applied.

6.7.63 RULE: *Exception constructor match.*

$$\frac{\begin{array}{l} E(\text{longvid}) = (en, \mathbf{e}) \quad v = (en, v') \\ E, v' \vdash \text{atpat} \Rightarrow VE/\text{FAIL} \end{array}}{E, v \vdash \text{longvid atpat} \Rightarrow VE/\text{FAIL}} \quad (146)$$

6.7.64 RULE: *Exception constructor match fail.*

$$\frac{E(\text{longvid}) = (en, \mathbf{e}) \quad v \notin \{en\} \times \text{Val}}{E, v \vdash \text{longvid atpat} \Rightarrow \text{FAIL}} \quad (147)$$

Any evaluation resulting in FAIL must do so because rule 134, rule 137, rule 145, or rule 147 has been applied.

6.7.65 RULE: *Reference pattern.*

$$\frac{s(a) = v \quad s, E, v \vdash \text{atpat} \Rightarrow VE/\text{FAIL}, s}{s, E, a \vdash \text{ref atpat} \Rightarrow VE/\text{FAIL}, s} \quad (148)$$

6.7.66 RULE: *Layered pattern.*

$$\frac{E, v \vdash \text{pat} \Rightarrow VE/\text{FAIL}}{E, v \vdash \text{vid as pat} \Rightarrow \{\text{vid} \mapsto (v, \mathbf{v})\} + VE/\text{FAIL}} \quad (149)$$

DYNAMIC SEMANTICS FOR MODULES

7.1. REDUCED SYNTAX

7.1.1 CLAUSE:

The syntax for Modules is reduced in the following manner, *in addition* to the reduced syntax made for Core.

7.1.2 CLAUSE: *Omit qualifications from descriptions.*

Qualifications “*of ty* ” are omitted from constructor and exception descriptions.

7.1.3 CLAUSE: *Omit sharing/where type qualifications.*

Any qualification “*sharing type* ...” on a specification, or “*where type* ...” on a signature expression, both are omitted.

7.2. COMPOUND OBJECTS

7.2.1 DEFINITION: *Compound objects.*

The compound objects for the Modules dynamic semantics, in addition to those for the Core dynamic semantics, are listed below:

$$\begin{aligned}
 (\textit{strid} : I, \textit{strexpr}, B) &\in \text{FunctorClosure} = (\text{StrId} \times \text{Int}) \times \text{StrExp} \times \text{Basis} \\
 I \text{ or } (SI, TI, VI) &\in \text{Int} = \text{StrInt} \times \text{TyInt} \times \text{ValInt} \\
 SI &\in \text{StrInt} = \text{StrId} \xrightarrow{\text{fin}} \text{Int} \\
 TI &\in \text{TyInt} = \text{TyCon} \xrightarrow{\text{fin}} \text{ValInt} \\
 VI &\in \text{ValInt} = \text{VId} \xrightarrow{\text{fin}} \text{IdStatus} \\
 G &\in \text{SigEnv} = \text{SigId} \xrightarrow{\text{fin}} \text{Int} \\
 F &\in \text{FunEnv} = \text{FunId} \xrightarrow{\text{fin}} \text{FunctorClosure} \\
 (F, G, E) \text{ or } B &\in \text{Basis} = \text{FunEnv} \times \text{SigEnv} \times \text{Env} \\
 (G, I) \text{ or } IB &\in \text{IntBasis} = \text{SigEnv} \times \text{Int}
 \end{aligned}$$

[Used in: 8.0.5]

7.2.2 DEFINITION: *Interface.*

An “**Interface**” $I \in \text{Int}$ represents a “view” of a structure.

7.2.3 CLAUSE: *Signatures evaluate to interfaces.*

Specifications and signature expressions will evaluate to interfaces; moreover, during the evaluation of a specification or signature expression, structures (to which a specification or signature expression may refer via datatype replicating specifications) are represented only by their interfaces.

7.2.4 DEFINITION: *Extracting an interface from an environment.*

To extract a value interface from a dynamic value environment we define the operation $\text{Inter} : \text{ValEnv} \rightarrow \text{ValInt}$ as follows:

$$\text{Inter}(VE) = \{vid \mapsto is ; VE(vid) = (v, is)\}$$

In other words, $\text{Inter}(VE)$ is the value interface obtained from VE by removing all values from VE .

We then extend Inter to a function $\text{Inter} : \text{Env} \rightarrow \text{Int}$ as follows:

$$\text{Inter}(SE, TE, VE) = (SI, TI, VI)$$

where $VI = \text{Inter}(VE)$ and

$$\begin{aligned} SI &= \{\text{strid} \mapsto \text{Inter } E ; SE(\text{strid}) = E\} \\ TI &= \{tycon \mapsto \text{Inter } VE' ; TE(tycon) = VE'\} \end{aligned}$$

7.2.5 DEFINITION: *Interface basis.*

An “**Interface Basis**” $IB = (G, I)$ is a value-free part of a basis, sufficient to evaluate signature expressions and specifications.

7.2.6 CLAUSE: *Extending Inter to create interface bases.*

The function Inter is extended to create an interface basis from a basis B as follows:

$$\text{Inter}(F, G, E) = (G, \text{Inter } E)$$

7.2.7 DEFINITION: *Cutting down an environment to an interface.*

A further operation

$$\downarrow : \text{Env} \times \text{Int} \rightarrow \text{Env}$$

is required, to cut down an environment E to a given interface I , representing the effect of an explicit signature ascription. We first define $\downarrow : \text{ValEnv} \times \text{ValInt} \rightarrow \text{ValEnv}$ by

$$VE \downarrow VI = \{vid \mapsto (v, is) ; VE(vid) = (v, is') \text{ and } VI(vid) = is\}$$

(Note that VE and VI need not have the same domain and that the identifier status is taken from VI .) We then define $\downarrow : \text{StrEnv} \times \text{StrInt} \rightarrow \text{StrEnv}$, $\downarrow : \text{TyEnv} \times \text{TyInt} \rightarrow \text{TyEnv}$ and $\downarrow : \text{Env} \times \text{Int} \rightarrow \text{Env}$ simultaneously as follows:¹

$$SE \downarrow SI = \{\text{strid} \mapsto E \downarrow I ; SE(\text{strid}) = E \text{ and } SI(\text{strid}) = I\}$$

$$TE \downarrow TI = \{tycon \mapsto VE' \downarrow VI' ; TE(tycon) = VE' \text{ and } TI(tycon) = VI'\}$$

$$(SE, TE, VE) \downarrow (SI, TI, VI) = (SE \downarrow SI, TE \downarrow TI, VE \downarrow VI)$$

7.2.8 CLAUSE: *Static elaboration naturally implements interfaces.*

It is important to note that an interface can also be obtained from the *static* value Σ of a signature expression; it is obtained by first replacing every type structure (θ, VE) in the range of every type environment TE by VE and then replacing each pair (σ, is) in the range of every value environment VE by is .

Thus in an implementation interfaces would naturally be obtained from the static elaboration; we choose to give separate rules here for obtaining them in the dynamic semantics since we wish to maintain our separation of the static and dynamic semantics, for reasons of presentation.

¹Note: there is a typo in the third equation $(SE, TE, VE) \downarrow (SI, TI, VI)$ which the Definition, as printed, reads erroneously as $(SE, TE, VE) \downarrow (SI, TE, VI)$.

7.3. INFERENCE RULES

7.3.1 DEFINITION: *Judgement form.*

The semantic rules allow sentences of the form

$$s, A \vdash phrase \Rightarrow A', s'$$

to be inferred, where A is either a basis, a signature environment, or empty; A' is some semantic object; and s, s' are states before and after the evaluation represented by the judgement.

7.3.2 DEFINITION: *Side-conditions.*

Some hypotheses in rules are not of this form; they are called “**side-conditions**”.

7.3.3 CONVENTION: *Options same as for Core.*

The conventions for options are the same as for Core (those in single brackets are all taken together, or none are taken together; separately, those in double brackets are all taken together, or none are taken).

7.3.4 CONVENTION: *State and exception conventions.*

The state convention (§6.7.4) and exception convention (§6.7.5) are adopted as in the Core dynamics. However, it may be shown that the only Modules phrases whose evaluation may cause a side-effect or generate an exception packet are of the form *strex*, *strdec*, *strbind* or *topdec*.

[Used in: 8.0.7]

Structure Expressions

$$B \vdash strexp \Rightarrow E/p$$

7.3.5 RULE: *Generative structure expression.*

$$\frac{B \vdash strdec \Rightarrow E}{B \vdash \text{struct } strdec \text{ end} \Rightarrow E} \quad (150)$$

7.3.6 RULE: *Structure long id.*

$$\frac{B(\text{longstrid}) = E}{B \vdash \text{longstrid} \Rightarrow E} \quad (151)$$

7.3.7 RULE: *Transparent signature constraint.*

$$\frac{B \vdash strexp \Rightarrow E \quad \text{Inter } B \vdash sigexp \Rightarrow I}{B \vdash strexp : sigexp \Rightarrow E \downarrow I} \quad (152)$$

7.3.8 RULE: *Opaque signature constraint.*

$$\frac{B \vdash strexp \Rightarrow E \quad \text{Inter } B \vdash sigexp \Rightarrow I}{B \vdash strexp :> sigexp \Rightarrow E \downarrow I} \quad (153)$$

7.3.9 RULE: *Functor application.*

$$\frac{\begin{array}{c} B(\text{funid}) = (strid : I, strexp', B') \\ B \vdash strexp \Rightarrow E \quad B' + \{strid \mapsto E \downarrow I\} \vdash strexp' \Rightarrow E' \end{array}}{B \vdash \text{funid } (strexp) \Rightarrow E'} \quad (154)$$

Before the evaluation of the functor body *strex*', the actual argument E is cut down by the formal parameter interface I , so that any opening of *strid* resulting from the evaluation of *strex*' will produce no more components than anticipated during the static elaboration.

7.3.10 RULE: *Let-structure expression.*

$$\frac{B \vdash \text{strdec} \Rightarrow E \quad B + E \vdash \text{strexpr} \Rightarrow E'}{B \vdash \text{let } \text{strdec} \text{ in } \text{strexpr} \text{ end} \Rightarrow E'} \quad (155)$$

Structure-level Declarations

$$\boxed{B \vdash \text{strdec} \Rightarrow E/p}$$

7.3.11 RULE: *Core declaration.*

$$\frac{E \text{ of } B \vdash \text{dec} \Rightarrow E'}{B \vdash \text{dec} \Rightarrow E'} \quad (156)$$

7.3.12 COMMENT: *For the “Core fragment” of ML.*

We can ignore the next two rules, and use these structure-level declarations rules to describe the dynamics of the “Core” fragment of Standard ML. See (§8.0.14).

7.3.13 RULE: *Structure declaration.*

$$\frac{B \vdash \text{strbind} \Rightarrow SE}{B \vdash \text{structure } \text{strbind} \Rightarrow SE \text{ in Env}} \quad (157)$$

7.3.14 RULE: *Local structure declaration.*

$$\frac{B \vdash \text{strdec}_1 \Rightarrow E_1 \quad B + E_1 \vdash \text{strdec}_2 \Rightarrow E_2}{B \vdash \text{local } \text{strdec}_1 \text{ in } \text{strdec}_2 \text{ end} \Rightarrow E_2} \quad (158)$$

7.3.15 RULE: *Empty declaration.*

$$\overline{B \vdash \Rightarrow \{\} \text{ in Env}} \quad (159)$$

7.3.16 RULE: *Sequential declaration.*

$$\frac{B \vdash \text{strdec}_1 \Rightarrow E_1 \quad B + E_1 \vdash \text{strdec}_2 \Rightarrow E_2}{B \vdash \text{strdec}_1 \langle ; \rangle \text{strdec}_2 \Rightarrow E_1 + E_2} \quad (160)$$

Structure Bindings

$$\boxed{B \vdash \text{strbind} \Rightarrow SE/p}$$

7.3.17 RULE: *Structure binding.*

$$\frac{B \vdash \text{strexpr} \Rightarrow E \quad \langle B \vdash \text{strbind} \Rightarrow SE \rangle}{B \vdash \text{strid} = \text{strexpr} \langle \text{and } \text{strbind} \rangle \Rightarrow \{\text{strid} \mapsto E\} \langle + SE \rangle} \quad (161)$$

Signature Expressions

$$\boxed{IB \vdash \text{sigexp} \Rightarrow I}$$

7.3.18 RULE: *Encapsulation signature expressions.*

$$\frac{IB \vdash \text{spec} \Rightarrow I}{IB \vdash \text{sig } \text{spec} \text{ end} \Rightarrow I} \quad (162)$$

7.3.19 RULE: *Signature identifier.*

$$\frac{IB(\text{sigid}) = I}{IB \vdash \text{sigid} \Rightarrow I} \quad (163)$$

Signature Declarations

$$\boxed{IB \vdash \text{sigdec} \Rightarrow G}$$

7.3.20 RULE: *Single signature declaration.*

$$\frac{IB \vdash \text{sigbind} \Rightarrow G}{IB \vdash \text{signature } \text{sigbind} \Rightarrow G} \quad (164)$$

Signature Bindings

$$\boxed{IB \vdash \text{sigbind} \Rightarrow G}$$

7.3.21 RULE: *Signature binding.*

$$\frac{IB \vdash \text{sigexp} \Rightarrow I \quad \langle IB \vdash \text{sigbind} \Rightarrow G \rangle}{IB \vdash \text{sigid} = \text{sigexp} \langle \text{and } \text{sigbind} \rangle \Rightarrow \{ \text{sigid} \mapsto I \} \langle + G \rangle} \quad (165)$$

Specifications

$$\boxed{IB \vdash \text{spec} \Rightarrow I}$$

7.3.22 RULE: *Value specification.*

$$\frac{IB \vdash \text{val } \text{valdesc} \Rightarrow VI \text{ in Int}}{\vdash \text{valdesc} \Rightarrow VI} \quad (166)$$

7.3.23 RULE: *Type specification.*

$$\frac{\vdash \text{typdesc} \Rightarrow TI}{IB \vdash \text{type } \text{typdesc} \Rightarrow TI \text{ in Int}} \quad (167)$$

7.3.24 RULE: *Equality type specification.*

$$\frac{\vdash \text{typdesc} \Rightarrow TI}{IB \vdash \text{eqtype } \text{typdesc} \Rightarrow TI \text{ in Int}} \quad (168)$$

7.3.25 RULE: *Datatype specification.*

$$\frac{\vdash \text{datdesc} \Rightarrow VI, TI}{IB \vdash \text{datatype } \text{datdesc} \Rightarrow (VI, TI) \text{ in Int}} \quad (169)$$

7.3.26 RULE: *Datatype replication specification.*

$$\frac{IB(\text{longtycon}) = VI \quad TI = \{ \text{tycon} \mapsto VI \}}{IB \vdash \text{datatype } \text{tycon} \text{ ==- datatype } \text{longtycon} \Rightarrow (VI, TI) \text{ in Int}} \quad (170)$$

7.3.27 RULE: *Exception specification.*

$$\frac{\vdash \text{exdesc} \Rightarrow VI}{IB \vdash \text{exception } \text{exdesc} \Rightarrow VI \text{ in Int}} \quad (171)$$

7.3.28 RULE: *Structure specification.*

$$\frac{IB \vdash \text{strdesc} \Rightarrow SI}{IB \vdash \text{structure } \text{strdesc} \Rightarrow SI \text{ in Int}} \quad (172)$$

7.3.29 RULE: *Include signature specification.*

$$\frac{IB \vdash \text{sigexp} \Rightarrow I}{IB \vdash \text{include } \text{sigexp} \Rightarrow I} \quad (173)$$

7.3.30 RULE: *Empty specification.*

$$\frac{}{IB \vdash \quad \Rightarrow \{ \} \text{ in Int}} \quad (174)$$

7.3.31 RULE: *Sequential specification.*

$$\frac{IB \vdash \text{spec}_1 \Rightarrow I_1 \quad IB + I_1 \vdash \text{spec}_2 \Rightarrow I_2}{IB \vdash \text{spec}_1 \langle ; \rangle \text{spec}_2 \Rightarrow I_1 + I_2} \quad (175)$$

Value Descriptions

$$\boxed{\vdash \text{valdesc} \Rightarrow VI}$$

7.3.32 RULE: *Value description.*

$$\frac{\langle \vdash \text{valdesc} \Rightarrow VI \rangle}{\vdash \text{vid} \langle \text{and } \text{valdesc} \rangle \Rightarrow \{\text{vid} \mapsto \mathbf{v}\} \langle +VI \rangle} \quad (176)$$

Type Descriptions

$$\boxed{\vdash \text{typdesc} \Rightarrow TI}$$

7.3.33 RULE: *Type description.*

$$\frac{\langle \vdash \text{typdesc} \Rightarrow TI \rangle}{\vdash \text{tyvarseq } \text{tycon} \langle \text{and } \text{typdesc} \rangle \Rightarrow \{\text{tycon} \mapsto \{\}\} \langle +TI \rangle} \quad (177)$$

Datatype Descriptions

$$\boxed{\vdash \text{datdesc} \Rightarrow VI, TI}$$

7.3.34 RULE: *Datatype description.*

$$\frac{\vdash \text{condesc} \Rightarrow VI \quad \langle \vdash \text{datdesc}' \Rightarrow VI', TI' \rangle}{\vdash \text{tyvarseq } \text{tycon} = \text{condesc} \langle \text{and } \text{datdesc}' \rangle \Rightarrow VI \langle +VI' \rangle, \{\text{tycon} \mapsto VI\} \langle +TI' \rangle} \quad (178)$$

Constructor Descriptions

$$\boxed{\vdash \text{condesc} \Rightarrow VI}$$

7.3.35 RULE: *Constructor description.*

$$\frac{\langle \vdash \text{condesc} \Rightarrow VI \rangle}{\vdash \text{vid} \langle \mid \text{condesc} \rangle \Rightarrow \{\text{vid} \mapsto \mathbf{c}\} \langle +VI \rangle} \quad (179)$$

Exception Descriptions

$$\boxed{\vdash \text{exdesc} \Rightarrow VI}$$

7.3.36 RULE:

$$\frac{\langle \vdash \text{exdesc} \Rightarrow VI \rangle}{\vdash \text{vid} \langle \text{and } \text{exdesc} \rangle \Rightarrow \{\text{vid} \mapsto \mathbf{e}\} \langle +VI \rangle} \quad (180)$$

Structure Descriptions

$$\boxed{IB \vdash \text{strdesc} \Rightarrow SI}$$

7.3.37 RULE: *Structure description.*

$$\frac{IB \vdash \text{sigexp} \Rightarrow I \quad \langle IB \vdash \text{strdesc} \Rightarrow SI \rangle}{IB \vdash \text{strid} : \text{sigexp} \langle \text{and } \text{strdesc} \rangle \Rightarrow \{\text{strid} \mapsto I\} \langle +SI \rangle} \quad (181)$$

Functor Bindings

$$\boxed{B \vdash \text{funbind} \Rightarrow F}$$

Note: we have fixed the typo reported by Rossberg [Ros18b].

7.3.38 RULE: *Functor binding*.

$$\frac{\text{Inter } B \vdash \text{sigexp} \Rightarrow I \quad \langle B \vdash \text{funbind} \Rightarrow F \rangle}{B \vdash \text{funid } (\text{strid} : \text{sigexp}) = \text{strex} \langle \text{and funbind} \rangle \Rightarrow \{ \text{funid} \mapsto (\text{strid} : I, \text{strex}, B) \} \langle + F \rangle} \quad (182)$$

Functor Declarations

$$\boxed{B \vdash \text{fundec} \Rightarrow F}$$

7.3.39 RULE: *Functor declaration*.

$$\frac{B \vdash \text{funbind} \Rightarrow F}{B \vdash \text{functor funbind} \Rightarrow F} \quad (183)$$

Top-level Declarations

$$\boxed{B \vdash \text{topdec} \Rightarrow B'/p}$$

7.3.40 RULE: *Structure-level declaration*.

Rossberg [Ros18b] correctly identifies a typo in the conclusion: the Definition writes “ $B' \langle ' \rangle$ ”, but what it means is really “ $B' \langle + B'' \rangle$ ”. We have fixed it here.

$$\frac{B \vdash \text{strdec} \Rightarrow E \quad B' = E \text{ in Basis} \quad \langle B + B' \vdash \text{topdec} \Rightarrow B'' \rangle}{B \vdash \text{strdec} \langle \text{topdec} \rangle \Rightarrow B' \langle + B'' \rangle} \quad (184)$$

7.3.41 RULE: *Signature declaration*.

Rossberg [Ros18b] correctly identifies a typo in the conclusion: the Definition writes “ $B' \langle ' \rangle$ ”, but what it means is really “ $B' \langle + B'' \rangle$ ”. We have fixed it here.

$$\frac{\text{Inter } B \vdash \text{sigdec} \Rightarrow G \quad B' = G \text{ in Basis} \quad \langle B + B' \vdash \text{topdec} \Rightarrow B'' \rangle}{B \vdash \text{sigdec} \langle \text{topdec} \rangle \Rightarrow B' \langle + B'' \rangle} \quad (185)$$

7.3.42 RULE: *Functor [top] declaration*.

Rossberg [Ros18b] correctly identifies a typo in the conclusion: the Definition writes “ $B' \langle ' \rangle$ ”, but what it means is really “ $B' \langle + B'' \rangle$ ”. We have fixed it here.

$$\frac{B \vdash \text{fundec} \Rightarrow F \quad B' = F \text{ in Basis} \quad \langle B + B' \vdash \text{topdec} \Rightarrow B'' \rangle}{B \vdash \text{fundec} \langle \text{topdec} \rangle \Rightarrow B' \langle + B'' \rangle} \quad (186)$$

PROGRAMS

8.0.1 DEFINITION: *Syntactic class of programs.*

The phrase class Program of programs is defined as follows

$$program ::= \text{topdec} ; \langle program \rangle$$

8.0.2 CLAUSE: *Inclusion of files is implementation-dependent.*

The Definition observes that sometimes the Standard ML compiler will be reading in a file, and at other times we will be working with a REPL. In either case, if an error is found, we should wish to avoid creating a distinction between, say, batch programs and interactive programs, and so the Definition tacitly regards all programs as interactive. This leaves it to implementers to clarify the inclusion of files.

Specifically, how including files affects updating the static and dynamic basis — this is implementation dependent behaviour.

8.0.3 CLAUSE: *Parsing error reporting is implementation dependent.*

How parsing errors are handled is left to the implementers (since it depends on the kind of parser employed).

8.0.4 CLAUSE: *Execution of programs.*

In this section, the “**Execution**” of a program means the combined elaboration and evaluation of the program.

8.0.5 DEFINITION: *Class of Basis, StaticBasis, DynamicBasis.*

The Definition wishes to disambiguate the usage of B for bases (since it was used as the metavariable for both static and dynamic bases). For ease of discussion, the Definition does the following:

- (1) In giving the semantics of programs, however, let us rename as StaticBasis the class Basis defined in the static semantics of modules (§5.1.2), Section 5.1, and let us use B_{STAT} to range over StaticBasis.
- (2) Let us rename as DynamicBasis the class Basis defined in the dynamic semantics of modules (§7.2.1), Section 7.2, and let us use B_{DYN} to range over DynamicBasis.
- (3) We now define

$$B \text{ or } (B_{\text{STAT}}, B_{\text{DYN}}) \in \text{Basis} = \text{StaticBasis} \times \text{DynamicBasis}.$$

8.0.6 DEFINITION: *Judgement forms.*

The Definition uses $\vdash_{\text{STAT}}$ for elaboration as defined in Section [Chapter] 5, and $\vdash_{\text{DYN}}$ for evaluation as defined in Section [Chapter] 7. Then $\vdash$ will be reserved for the execution of programs, which thus is expressed by a sentence of the form

$$s, B \vdash program \Rightarrow B', s'$$

This may be read as follows: starting in basis B with state s the execution of *program* results in a basis B' and a state s' .

8.0.7 CLAUSE: *Programs never result in exceptions.*

Executing a program never results in an exception. If the evaluation of a *topdec* yields an exception (for instance because of a **raise** expression), then the result of executing the program “*topdec* ;” is the original basis together with the state which is in force when the exception is generated.

In particular, this means the exception convention (§7.3.4) is not applicable to the rules the Definition describes in this section [chapter].

8.0.8 CONVENTION: *Non-elaboration of top-level declaration.*

The Definition represents the non-elaboration of a top-level declaration by “ $\dots \vdash_{\text{STAT}} \text{topdec} \not\Rightarrow$ ”.

Programs

$$s, B \vdash \text{program} \Rightarrow B', s'$$

8.0.9 RULE: *Failing elaboration.*

$$\frac{B_{\text{STAT}} \text{ of } B \vdash_{\text{STAT}} \text{topdec} \not\Rightarrow \quad \langle s, B \vdash \text{program} \Rightarrow B', s' \rangle}{s, B \vdash \text{topdec} ; \langle \text{program} \rangle \Rightarrow B\langle' \rangle, s\langle' \rangle} \quad (187)$$

A failing elaboration has no effect whatever.

8.0.10 COMMENT: *Implications of negative premise in rule 187.*

Rossberg [Ros18b] observes the negative premise in rule 187 has unfortunate implications.

If we interpret the rule strictly, it precludes any conforming implementation from proving any sort of conservative semantic extension to the language. Any extension that allows declarations to elaborate that would be illegal according to the Definition (e.g., polymorphic records) can be observed through this rule and change the behaviour of consecutive declarations. This could alter the observed behaviour of a program. Consider Rossberg’s example:

```
val s = "no";
strdec
val s = "yes";
print s;
```

where the *strdec* only elaborates if some extension is supported. In that case, the program will print **yes**, otherwise it will print **no**.

Rossberg interprets this as suggesting that formalizing an interactive toplevel is not worth the trouble. He’s probably right.

8.0.11 RULE: *Evaluation raising exception.*

$$\frac{B_{\text{STAT}} \text{ of } B \vdash_{\text{STAT}} \text{topdec} \Rightarrow B_{\text{STAT}}^{(1)} \quad s, B_{\text{DYN}} \text{ of } B \vdash_{\text{DYN}} \text{topdec} \Rightarrow p, s' \quad \langle s', B \vdash \text{program} \Rightarrow B', s'' \rangle}{s, B \vdash \text{topdec} ; \langle \text{program} \rangle \Rightarrow B\langle' \rangle, s'\langle' \rangle} \quad (188)$$

An evaluation which yields an exception nullifies the change in the static basis, but does not nullify side-effects on the state which may have occurred before the exception was raised.

8.0.12 RULE: *Success.*

$$\frac{B_{\text{STAT}} \text{ of } B \vdash_{\text{STAT}} \text{topdec} \Rightarrow B_{\text{STAT}}^{(1)} \quad s, B_{\text{DYN}} \text{ of } B \vdash_{\text{DYN}} \text{topdec} \Rightarrow B_{\text{DYN}}^{(1)}, s' \quad B' = B \oplus (B_{\text{STAT}}^{(1)}, B_{\text{DYN}}^{(1)}) \quad \langle s', B' \vdash \text{program} \Rightarrow B'', s'' \rangle}{s, B \vdash \text{topdec} ; \langle \text{program} \rangle \Rightarrow B'\langle' \rangle, s'\langle' \rangle} \quad (189)$$

8.0.13 COMMENT: *Ambiguity in semicolons.*

Recall (§3.6.7) “No *topdec* may contain, as an initial segment, a *strdec* followed by a semicolon.” Kahr [Kah93, §8.3] and Rossberg [Ros18b] both point out this is a problem.

Kahr points out that “*dec*₁;*dec*₂;*dec*₃” can be parsed in two different ways (at least), taking *dec*₁₂ as *dec*₁;*dec*₂ and *dec*₂₃ as *dec*₂;*dec*₃, these evaluate to the same thing provided in rule 24 and rule 123 we can have

$$C \oplus (E_1 \oplus E_2) = (C \oplus E_1) \oplus E_2.$$

Unfortunately, this does not hold, and the right-hand side of this equation can contain more type names than the one of the left-hand side. This means that the associativity of $\langle ; \rangle$ is not obvious.

Rossberg resolves this problem by having HaMLet’s parser reduce to *strdec* as soon as possible (since this solves other problems as well). This seems to be the “spirit of the Law”, even if the “letter of the Law” leaves much to be desired.

8.0.14 DEFINITION: *Core language programs.*

A program is called a “**Core Language Program**” if it can be parsed in the reduced grammar defined as follows:

- (1) Replace the definition of top-level declarations by

$$\textit{topdec} ::= \textit{strdec}$$

- (2) Replace the definition of structure-level declarations by

$$\textit{strdec} ::= \textit{dec}$$

[Used in: 7.3.12]

8.0.15 PUZZLE: *Do first-class structures simplify things?*

Russo [Rus98, Rus00] and later inspired by this Rossberg [Ros18a] proposed first-class structures (modules, signatures, functors) for Standard ML. Since facilitating the second-class nature of the module system complicates things greatly, do we simplify life by making modules first-class?

Bibliography

- [Kah93] Stefan Kahrs, *Mistakes and Ambiguities in the definition of Standard ML*, LFCS Report ECS-LFCS-93-257, University of Edinburgh, April 1993, Available at <http://www.cs.kent.ac.uk/pubs/1993/569>. ix, 13, 20, 56, 77
- [Kah96] ———, *Mistakes and Ambiguities in the definition of Standard ML — Addenda*, <http://www.cs.kent.ac.uk/~smk/errors-new.ps.Z>, 1996. ix
- [Ken24] Alan U Kennington, *Differential geometry reconstructed: A unified systematic framework*, 2024, draft available <http://www.topology.org/tex/conc/dgstats.php>. viii
- [MT91] Robin Milner and Mads Tofte, *Commentary on Standard ML*, MIT press, 1991. vii, viii, 27, 57
- [MTH90] Robin Milner, Mads Tofte, and Robert Harper, *The Definition of Standard ML*, MIT press, 1990. vii
- [MTHM97] Robin Milner, Mads Tofte, Robert Harper, and David MacQueen, *The Definition of Standard ML: Revised*, MIT press, 1997. vii, 11
- [Pie05] Benjamin C Pierce, *Advanced Topics in Types and Programming Languages*, MIT press, 2005. 79
- [PR05] François Pottier and Didier Rémy, *The Essence of ML Type Inference*, in Pierce [Pie05], Extended version available <http://crystal.inria.fr/attapl/emlti-long.pdf>, pp. 389–489. 32
- [Ros18a] Andreas Rossberg, *1ML—Core and Modules United*, Journal of Functional Programming **28** (2018), e22. 77
- [Ros18b] ———, *Defects in the Revised Definition of Standard ML*, <https://people.mpi-sws.org/~rossberg/papers/sml-defects-2013-09-18.pdf>, 2018. ix, 20, 24, 26, 56, 73, 76, 77
- [Rus98] Claudio V Russo, *Types for Modules*, Ph.D. thesis, University of Edinburgh, 1998. 77
- [Rus00] ———, *First-class structures for Standard ML*, European Symposium on Programming, Springer, 2000, pp. 336–350. 77
- [SJ23] Elizabeth Scott and Adrian Johnstone, *Analysing the SML97 Definition: Lexicalisation*, Eelco Visser Commemorative Symposium (EVCS 2023), Schloss-Dagstuhl-Leibniz Zentrum für Informatik, 2023. 26
- [SJW23] Elizabeth Scott, Adrian Johnstone, and Robert Walsh, *Multiple input parsing and lexical analysis*, ACM Transactions on Programming Languages and Systems **45** (2023), no. 3, 1–44. 19

Index

$A \xrightarrow{\text{fin}} B$, 4.2.2
`type(scon)` , 4.1.6
Fin A , 4.2.1
`val(scon)` , 6.2.5
 $f + g$, 4.2.4
59.2, 7.3.1

Admits equality, 4.4.1, 4.4.5

Equality
 Type, 4.4.1
 Type function, 4.4.5

Fin A , 4.2.1

Finite map, 4.2.2
 modification, 4.2.4

Pattern
 Exhaustive, 4.11.3
 Irredundant, 4.11.3

Special constant
 Type, 4.1.6

Type
 Equality —, 4.4.1
 of special constants, 4.1.6

Type function, 4.4.2
 beta reduction, 4.4.6
 Equality —, 4.4.5

Type scheme
 generalizes a type, 4.5.1
`type(scon)` , 4.1.6

Summary Statistics:

- Total items: 429
- clauses 78
- comments 29
- conventions 16
- definitions 81
- deviations 2
- examples 4
- grammars 10
- inference-rules 189
- puzzles 5
- remarks 14
- theorems 1